



Ministerul Educației și Cercetării
Universitatea "OVIDIUS" Constanța
Facultatea de Matematică și Informatică
Specializarea Informatică

Achieve+: Sistem software pentru învățare online și predicția performanței școlare

Lucrare de licență

Coordonator științific:
Conf. univ. dr. Băutu Elena

Absolvent:
Ivan Alexandra

Constanța
2025

Cuprins

Cuprins	i
Lista Figurilor	iv
1 Introducere	3
1.1 Motivația temei alese	3
1.2 Conținut	4
2 Starea actuală a domeniului	5
2.1 Studii și articole științifice	5
2.2 Aplicații dedicate învățării online	8
2.2.1 Aplicația Duolingo	8
2.2.2 Khan Academy	9
2.2.3 Coursera	10
2.2.4 Analiză comparativă	11
3 Tehnologii folosite	12
3.1 Mediul de dezvoltare integrat - Visual Studio	12
3.2 Platforma și limbajele de programare	12
3.2.1 .NET 8	13

3.2.2	C#	13
3.2.3	HTML	13
3.2.4	CSS	13
3.2.5	JavaScript	14
3.2.6	Python și Flask	14
3.3	Arhitectură și framework-uri	14
3.3.1	Arhitectura MVC	14
3.3.2	ASP.NET Core	16
3.3.3	Entity Framework Core	16
3.3.4	Bootstrap	16
3.4	Baza de date	16
3.4.1	Microsoft SQL Server	17
3.4.2	SQL	17
3.5	Algoritmi și metode folosite	17
3.5.1	BCrypt	17
3.5.2	Algoritmul Cosine Similarity	18
3.5.3	Algoritmul de predicție	19
4	Proiectarea și dezvoltarea aplicației Achieve+	21
4.1	Cerințe funcționale	21
4.2	Cerințe nefuncționale	22
4.3	Definirea cazurilor de utilizare	23
4.3.1	Cazul de utilizare Adăugare Lecție	23
4.3.2	Cazul de utilizare Parcurgere Lecție	24
4.3.3	Cazul de utilizare Administrare Test	24
4.3.4	Cazul de utilizare Câștigare badges	25
4.4	Diagramele de modelare	25
4.4.1	Diagrama cazurilor de utilizare	25

4.4.2	Diagrama de activitate	26
4.4.3	Diagrama de secvențe	27
4.4.4	Diagrama de secvențe în proiectare	29
4.5	Proiectarea bazei de date	30
4.5.1	Restricții	30
4.5.2	Descrierea entităților	32
4.5.3	Modelul MRD	33
4.6	Implementarea funcționalităților	37
4.6.1	Register	37
4.6.2	Login	39
4.6.3	Resetarea parolei	41
4.6.4	Recomandarea lecțiilor	44
4.6.5	Predicția performanței	49
5	Prezentarea aplicației	51
5.1	Utilizarea aplicației	51
5.1.1	Student	54
5.1.2	Profesor	60
6	Concluzii	68
	Referințe bibliografice	69

Lista Figurilor

2.1	Capturi din aplicația Duolingo	9
2.2	Captură din platforma Khan Academy	10
2.3	Captură din platforma Coursera	11
2.4	Tabel comparativ	11
3.1	Cum funcționează MVC: de la cererea utilizatorului la răspunsul aplicației . . .	15
4.1	Diagrama cazurilor de utilizare	26
4.2	Diagrama de activitate - Parcurgere lecție	26
4.3	Diagrama de activitate - Adăugare lecție	27
4.4	Diagrama de secvențe - Adăugare lecție	28
4.5	Diagrama de secvențe - Parcurgere lecție	28
4.6	Diagrama de secvențe în proiectare - Adăugare lecție	29
4.7	Diagrama de secvențe în proiectare - Parcurgere lecție	29
4.8	Diagrama Entitate-Asociere (ERD) pentru aplicația Achieve+	30
4.9	Validarea modelului	37
4.10	Validarea datelor introduse	37
4.11	Verificarea existenței unui utilizator	38
4.12	Crearea utilizatorului și a profilului asociat	38
4.13	Redirecționarea utilizatorului	39

4.14	Tratarea erorilor	39
4.15	Validarea câmpurilor de autentificare	40
4.16	Verificarea utilizatorului și a parolei	40
4.17	Actualizarea datelor și inițializarea sesiunii	41
4.18	Redirecționarea și tratarea erorilor	41
4.19	Validarea câmpurilor pentru resetarea parolei	42
4.20	Verificarea parolelor	42
4.21	Căutarea utilizatorului după email	43
4.22	Resetarea parolei și tratarea excepțiilor	43
4.23	Generarea vectorului pe baza lecțiilor începute	44
4.24	Generarea vectorului pe baza lecțiilor începute	44
4.25	Generarea vectorului pe baza lecțiilor neîncepute	45
4.26	Selectarea celor mai relevante lecții	46
4.27	Lecțiile neîncepute	47
4.28	Funcția de normalizare	48
4.29	Funcția similitudinii cosinus	48
4.30	Comunicarea cu serviciul extern	49
4.31	Metoda "BuildFeaturesFromDb"	49
4.32	Metoda "BuildFeaturesFromDb" - continuare	50
4.33	Antrenarea modelelor pentru predicție	50
4.34	Modelul PredictionResponse	50
5.1	Prima parte a paginii principale	51
5.2	A doua parte a paginii principale	52
5.3	Pagina lecțiilor, fără autentificare	52
5.4	Pagina modulelor, fără autentificare	52
5.5	Pagina de Register	53
5.6	Pagina de Login	53

5.7	Pagina Dashboard	54
5.8	Pagina modulelor	55
5.9	Detaliile unui modul	55
5.10	Pagina lecțiilor	55
5.11	Detaliile unei lecții	56
5.12	Pagina testelor	56
5.13	Conținutul unui test	56
5.14	Finalizarea unui test	57
5.15	Pagina temelor	57
5.16	Detaliile unei teme	57
5.17	Pagina insignelor	58
5.18	Pagina progresului	58
5.19	Pagina progresului	58
5.20	Pagina progresului	59
5.21	Pagina progresului	59
5.22	Raportul unui student	59
5.23	Pagina Dashboard pentru profesori	60
5.24	Pagina modulelor din perspectiva profesorului	60
5.25	Crearea unui modul	61
5.26	Modificarea unui modul	61
5.27	Ștergerea unui modul	62
5.28	Pagina lecțiilor din perspectiva profesorului	62
5.29	Crearea unei lecții	62
5.30	Modificarea unei lecții	63
5.31	Ștergerea unei lecții	63
5.32	Pagina temelor din perspectiva profesorului	63
5.33	Crearea unei teme	64

5.34	Modificarea unei teme	64
5.35	Ștergerea unei teme	64
5.36	Vizualizarea detaliilor unei teme	65
5.37	Vizualizarea testelor din perspectiva profesorului	65
5.38	Crearea unui test	65
5.39	Modificarea unui test	66
5.40	Ștergerea unui test	66
5.41	Pagina de progres din perspectiva profesorului	66
5.42	Raportul clasei	67
5.43	Raportul unui student	67

Rezumat

Lucrarea de licență **Achieve+: Sistem software pentru învățare online și predicția performanței școlare** propune dezvoltarea unei aplicații educaționale moderne, care combină lecții interactive, testare dinamică și algoritmi de recomandare personalizați. Platforma este construită folosind tehnologii precum ASP.NET Core, Entity Framework, JavaScript și SQL Server, urmând arhitectura MVC. Printre funcționalitățile cheie se numără înregistrarea utilizatorilor, parcurgerea de lecții, testarea cunoștințelor, câștigarea de badge-uri, precum și recomandarea de conținut și predicția performanței școlare pe baza istoricului. Sistemul se adresează atât elevilor, cât și profesorilor, oferind interfețe și funcționalități dedicate fiecărui rol. Algoritmi precum Cosine Similarity sunt utilizați pentru recomandări, iar securitatea este asigurată prin autentificare și criptarea parolelor. Achieve+ reprezintă o soluție completă și scalabilă pentru susținerea învățării personalizate în mediul online.

Abstract

The thesis **Achieve+: Software System for Online Learning and School Performance Prediction** proposes the development of a modern educational application that combines interactive lessons, dynamic testing, and personalized recommendation algorithms. The platform is built using technologies such as ASP.NET Core, Entity Framework, JavaScript, and SQL Server, following the MVC architecture. Key features include user registration, lesson browsing, knowledge testing, badge earning, as well as content recommendation and performance prediction based on user history. The system targets both students and teachers, providing dedicated interfaces and functionalities for each role. Algorithms such as Cosine Similarity are used for recommendations, and security is ensured through authentication and password encryption. Achieve+ offers a complete and scalable solution to support personalized learning in the online environment.

Capitolul 1

Introducere

1.1 Motivația temei alese

De-a lungul timpului, educația a fost fundamentală pentru o societate stabilă și funcțională, iar digitalizarea acestei sfere a contribuit la o creștere semnificativă a acumulării de cunoștințe, întrucât, în ziua de azi, suntem înconjurați de informație. Învățarea online a revoluționat modul de studiu, permițând studenților să învețe de oriunde, în ritmul propriu.

Un avantaj al platformelor de învățare online (E-Learning) este că majoritatea sunt gratuite, oferind persoanelor în condiții defavorizate acces la cursuri din multiple domenii, doar accesând un site. Aceste platforme livrează o gamă largă de materiale de studiu din domenii variate și vaste, acordând informații suplimentare dintr-un domeniu inaccesibil la bibliotecă sau la o universitate. Însă, una dintre cele mai dificile probleme ale acestor platforme este monitorizarea în timp real a progresului cursanților, incluzând depistarea abandonului sau a lipsei de interes față de domeniul respectiv. Astfel, elaborarea unui sistem software care îmbină învățarea online cu algoritmi de predicție a performanței școlare reprezintă o adevărată necesitate în vremurile actuale.

O altă provocare ar fi lipsa interacțiunii în timp real cu profesorii și cu alți studenți, alături de monitorizarea evoluției fiecărui cursant în parte. În plus, majoritatea platformelor nu au implementate modalități de oferire a feedback-ului personalizat și recomandări specifice și individuale, factor care pot crește demotivarea și abandonul.

Un software inteligent și antrenat poate depista cursanții în dificultate, le poate recomanda metode de sprijin și poate ajusta conținutul și dificultatea materialelor în funcție de nevoile fiecărui student. Prin asigurarea unui sprijin personalizat, studenții își pot găsi motivația să își continue studiile. Un astfel de sistem, eficient pentru predicția performanței școlare, ar putea utiliza diferite date precum prezența la cursuri, notele de la testările anterioare, interacțiunea cu platformele de E-Learning. În acest mod, se pot emite sugestii personalizate pentru îmbunătățirea performanțelor academice.

Fiind o persoană curioasă și dornică de a afla în permanență lucruri noi, consider că o

aplicație cu aceste implementări ar fi extrem de utilă, atât în medii academice precum școli, licee, universități, cât și în medii non-formale, pentru studierea diferitelor cursuri de acasă.

1.2 Conținut

Lucrarea de față urmărește dezvoltarea unui sistem software pentru învățare online și predicția performanței școlare. În cele ce urmează, voi sintetiza principalele idei din fiecare capitol, în scopul accentuării fiecărei etape din realizarea acestei aplicații.

În capitolul 1 sunt descrise motivația alegerii acestei teme, iar următorul capitol cuprinde o analiză a stării actuale a domeniului, prezentând studii relevante și aplicații din zona educațională, alături de o analiză comparativă a acestora.

Capitolul 3 își propune detalierea procesului de proiectare și dezvoltare a aplicației, pornind de la cerințele funcționale și nefuncționale, descrierea cazurilor de utilizare, diagramele UML modelate, baza de date și implementarea funcționalităților cheie. Capitolul 4 oferă informații despre tehnologiile folosite, atât pentru partea de interfață, cât și algoritmii testați, arhitectura și framework-urile utilizate. În final, capitolul 5 demonstrează funcționalitățile aplicației din perspectiva ambelor roluri (student și profesor), ilustrând cum platforma permite gestionarea lecțiilor, modulelor, temelor, testelor și a progresului individual.

Capitolul 2

Starea actuală a domeniului

În acest capitol, voi prezenta starea actuală a domeniului inteligenței artificiale și al învățării automate utilizate în educație, printr-o evaluare a studiilor recente și a realizărilor din acest domeniu. Capitolul este divizat în două subcapitole: în primul, mă voi concentra pe explorarea studiilor și articolelor științifice relevante, care oferă o privire de ansamblu asupra progreselor realizate și a impactului acestor tehnologii asupra educației. În al doilea subcapitol, voi include diverse aplicații pentru învățarea online, pentru a scoate în evidență diferențele, asemănările și potențialul de dezvoltare ale tehnologiilor aplicate în educație.

2.1 Studii și articole științifice

Studiile recente subliniază tot mai frecvent importanța utilizării inteligenței artificiale și a metodelor de învățare automată în mediul academic. Numeroase studii explorează modalitățile prin care aceste tehnologii pot transforma procesul educațional, prin predicția performanței studenților și îmbunătățirea sistemelor de învățare online.

Analiza literaturii de specialitate este crucială pentru înțelegerea realizărilor actuale și determinarea direcțiilor viitoare de cercetare. În această secțiune, voi prezenta o selecție de studii relevante, fiecare abordând teme esențiale în educația digitală, de la metode de predicție a performanței academice până la optimizarea platformelor de învățare online.

În articolul [1], publicat în anul 2024, autorii analizează aplicarea tehnicilor de învățare automată (machine learning) pentru prezicerea probabilității de absolvire a studenților. Acest studiu reprezintă o sintetizare a concluziilor a 70 de articole pe această temă, publicate între 2018 și 2023.

Rezultatele cercetării indică faptul că modelele de inteligență artificială pot anticipa finalizarea studiilor de către studenți cu o acuratețe de până la 90%, folosind date provenite din sistemele de management al învățării (LMS) și din baze de date academice. Algoritmii de învățare automată care se dovedesc a fi cei mai eficace cuprind Suport Vector Machine (SVM) și Random Forest, având capacități superioare de analiză a modelelor complexe și

depistarea studenților cu risc de abandon.

Un alt aspect esențial punctat în articol este nevoia dezvoltării unor sisteme de depistare timpurie, care să recunoască studenții cu probleme academice și să asigure sprijin personalizat. Articolul prezintă o perspectivă amănunțită asupra domeniului, autorii scoțând în evidență constrângerile studiilor precedente, precum lipsa unor criterii standard pentru evaluarea modelelor de predicție și greutățile utilizării acestora în diverse medii educaționale.

Autorii articolului [2] cercetează utilizarea algoritmilor de machine learning supervizat pentru predicția performanței academice a studenților. Acest studiu se bazează pe date demografice, comportamentale și educaționale ale studenților de la Universitatea din Basra, adunate în perioada 2017-2019, pentru descoperirea celor mai eficiente modele de predicție.

Din rezultatele studiului reiese faptul că algoritmul Logistic Regression prezintă cea mai mare acuratețe (88,8% pentru promovare, 68,7% pentru notele finale), comparativ cu alte modele precum Decision Tree, Naive Bayes și Neural Network. Cercetarea scoate în evidență impactul volumului setului de date și al alegerii variabilelor asupra preciziei predicțiilor, utilizând software-ul Weka. [3]

Studiul prezintă eficiența machine learning-ului în educație, însă evidențiază și constrângeri precum provocările legate de colectarea datelor și generalizarea rezultatelor pentru sisteme educaționale diferite. Acest articol este relevant pentru lucrarea de față întrucât se concentrează pe utilizarea inteligenței artificiale în cadrul dezvoltării unui sistem software educațional.

În cadrul articolului [4], autorii introduc un model bazat pe deep learning destinat predicției performanței academice a studenților, numit GritNet, bazat pe activitatea lor în timpul cursurilor online. Acest model folosește o arhitectură bazată pe rețele neuronale bidirecționale de tip Long Short-Term Memory (BLSTM), având capacitatea de a procesa date brute, eliminând nevoia analizării manuale a caracteristicilor. GritNet a fost testat folosind date reale provenite de la platforma Udacity, dovedind o acuratețe superioară în comparație cu metodele tradiționale fundamentate pe regresie logică, mai ales în primele săptămâni ale cursului, atunci când predicțiile reprezintă o mare provocare.

Acest model se bazează pe procesarea secvențială a lucrului studenților, precum citirea materialelor, rezolvarea testelor și predarea proiectelor, utilizând Global Max Pooling (GMP) pentru determinarea celor mai relevante părți ale acestor secvențe. Acest aspect permite modelului să efectueze predicții precise, chiar și în cazul unor date dezechilibrate. Rezultatele acestui studiu arată că GritNet oferă o îmbunătățire semnificativă a acurateței încă din primele săptămâni ale cursului, permițând intervenții educaționale mai rapide și mai eficiente.

Astfel, GritNet ilustrează o abordare inovatoare în domeniul predicției performanței școlare, prezentând o soluție eficientă și scalabilă pentru prezicerea rezultatelor academice. Modelul acesta poate fi inclus în diverse platforme de învățare online pentru a oferi sprijin studenților și a reduce rata de abandon, asistând la îmbunătățirea procesului de învățare.

Autorii articolului [5] analizează cât de bine funcționează metodele de învățare sincrone și asincrone în ceea ce privește performanța academică a studenților. Acest studiu reprezintă

o meta-analiză care sintetizează rezultatele a 14 studii publicate între anii 2002 și 2022, incluzând un număr de 1908 participanți în cadrul învățării sincrone și 804 în cadrul celei asincrone. Principala concluzie a acestui studiu este că învățarea asincronă tinde să fie ușor mai eficientă în îmbunătățirea cunoștințelor studenților comparativ cu învățarea sincronă, deși diferența este nesemnificativă.

Studiul a investigat, de asemenea, posibilele influențe cauzate de nivelul educațional și domeniul de studiu. Rezultatele au indicat faptul că efectul învățării asincrone rămâne destul de constant pe diferite niveluri educaționale (grădiniță, gimnaziu, învățământ superior, formare profesională), însă a scos în evidență un aspect conform căruia învățarea asincronă ar putea fi mai benefică pentru discipline precum matematica. Matematica necesită un proces cognitiv mai activ, iar învățarea asincronă le permite studenților să își gestioneze singuri ritmul în care asimilează toate cunoștințele. În plus, alte avantaje ale acestui model constau în flexibilitatea organizării studiului și oportunitatea accesării materialelor de curs în orice moment. Cu toate acestea, învățarea sincronă continuă să fie esențială pentru interacțiunea în mod direct cu profesorii și colegii, asigurând suport în timp real și un nivel mai ridicat de implicare.

Deși articolul evidențiază avantajele ambelor metode, autorii pun accent pe necesitatea îmbinării acestora pentru a optimiza eficiența procesului educațional.

În articolul [6], autorii analizează aspectele care influențează implicarea studenților în învățarea online. Acest studiu a fost realizat la Universitatea din Amsterdam, bazându-se pe grupuri de studenți din domeniul științelor sociale. Studiul subliniază trei aspecte ale implicării studenților în mediul online: comportamental (atenție și efort), afectiv (interacțiuni și sentiment de apartenență) și cognitiv (procesare și aprofundare a cunoștințelor).

Rezultatele indică faptul că activitățile online care stimulează interacțiunea și colaborarea cresc implicarea studenților. Printre metodele cele mai eficiente se numără folosirea sălilor de discuție (breakout rooms), sondajele online și exercițiile interactive. În plus, feedback-ul clar și o structură bine organizată a cursurilor sunt cruciale pentru menținerea interesului studenților. Studiul subliniază că învățarea sincronă contribuie la dezvoltarea sentimentului de apartenență, în timp ce învățarea asincronă oferă flexibilitate și independență în procesul de învățare.

Deși acest articol nu abordează în mod direct predicția performanței școlare, acesta aduce contribuții semnificative în dezvoltarea unui sistem software de învățare online, sugerând că includerea unor elemente interactive și a unei comunicări eficiente pot îmbunătăți experiența educațională. În plus, se evidențiază aspectele care influențează realizările studenților în mediul digital, date ce pot fi utilizate pentru optimizarea platformelor educaționale și personalizarea procesului de învățare.

În cadrul articolului [7], autorii propun un sistem avansat de recomandare a cursurilor destinate platformelor educaționale online. Studiul se concentrează pe problema suprasaturării informaționale din sistemele MOOC (Massive Open Online Courses), unde utilizatorii întâmpină dificultăți în alegerea cursurilor adecvate. Pentru a spori experiența studenților, autorii dezvoltă un model bazat pe Deep Reinforcement Learning (DRL) și o strategie multi-agent pentru recomandarea personalizată a cursurilor.

Modelul propus, denumit DRR (Deep Reinforcement Recommender), folosește un model Actor-Critic pentru a aborda recomandările ca pe un proces decizional secvențial. Algoritmul analizează stilul de învățare, abilitățile, preferințele și nivelul adaptabilității studentului, generând sugestii personalizate pentru cursuri. Experimentele efectuate pe un set de date MOOC, care conține 100.000 de recenzii ale cursurilor Coursera, arată că DRR depășește modelele tradiționale, precum Collaborative Filtering, Matrix Factorization și DQN (Deep Q-Network), având un grad mai mare de precizie în recomandări.

Chiar dacă acest studiu pune accent pe dezvoltarea procesului de recomandare a cursurilor, el este relevant în cadrul dezvoltării unui software de învățare online, deoarece ilustrează modul în care tehnicile de inteligență artificială pot fi utilizate pentru personalizarea experienței educaționale. Implementarea unui astfel de sistem într-o platformă de învățare online ar putea ajuta la îmbunătățirea parcursului educațional al studenților și la sporirea ratei de finalizare a cursurilor.

2.2 Aplicații dedicate învățării online

În prezent, platformele de învățare online iau amploare din ce în ce mai mult, cursurile acestora fiind acreditate pe plan internațional. În acest subcapitol voi realiza o analiză pentru trei platforme educaționale populare, care folosesc inteligență artificială, machine learning și alte tehnologii avansate. Compararea acestora va sublinia funcționalitățile fiecărei platforme, oferind o perspectivă mai clară asupra modului în care tehnologia poate inova educația.

2.2.1 Aplicația Duolingo

Duolingo¹ este una dintre cele mai populare aplicații pentru învățarea limbilor străine, oferind o abordare prin jocuri și accesibilă tuturor utilizatorilor, oricare ar fi vârsta acestora. Aplicația integrează inteligența artificială și machine learning pentru adaptarea lecțiilor în funcție de progresul fiecărui utilizator, examinând date precum rata de reușită a exercițiilor, frecvența utilizării și erorile făcute de utilizator.

Un aspect distinctiv al aplicației Duolingo este sistemul de recompense, în care utilizatorii sunt puși într-o ligă, practic într-o competiție cu ceilalți. Acest lucru motivează utilizatorii să se antreneze constant și să primească premii virtuale. Cu toate că aplicația nu include un model avansat de predicție a performanței academice, utilizează adaptive learning, ajustând dificultatea exercițiilor pe baza progresului individual.

¹ Aplicația mobilă Duolingo: <https://www.duolingo.com/>, accesat la 14.03.2025



Figura 2.1: Capturi din aplicația Duolingo

2.2.2 Khan Academy

Khan Academy² este o platformă de învățare online gratuită care oferă cursuri organizate în domenii precum matematica, științe, informatică și economie. Această platformă integrează inteligența artificială și analiza predicției pentru monitorizarea progresului utilizatorilor și oferă exerciții personalizate în funcție de nivelul de cunoștințe și ritmul de învățare. Un element distinctiv este panoul de progres, unde atât elevii, cât și profesorii pot analiza rapoarte detaliate despre performanță, ceea ce permite ajustarea modelelor de studiu. Deși nu are elemente de gamificare precum Duolingo, Khan Academy oferă insigne și niveluri pentru motivarea utilizatorilor să continue să învețe. Comparativ cu alte aplicații de învățare, aceasta se dovedește a fi mai potrivită pentru educația formală, fiind frecvent utilizată în școli pentru a sprijini procesul de predare.

² Platforma Khan Academy: <https://www.khanacademy.org/>, accesat la 14.03.2025

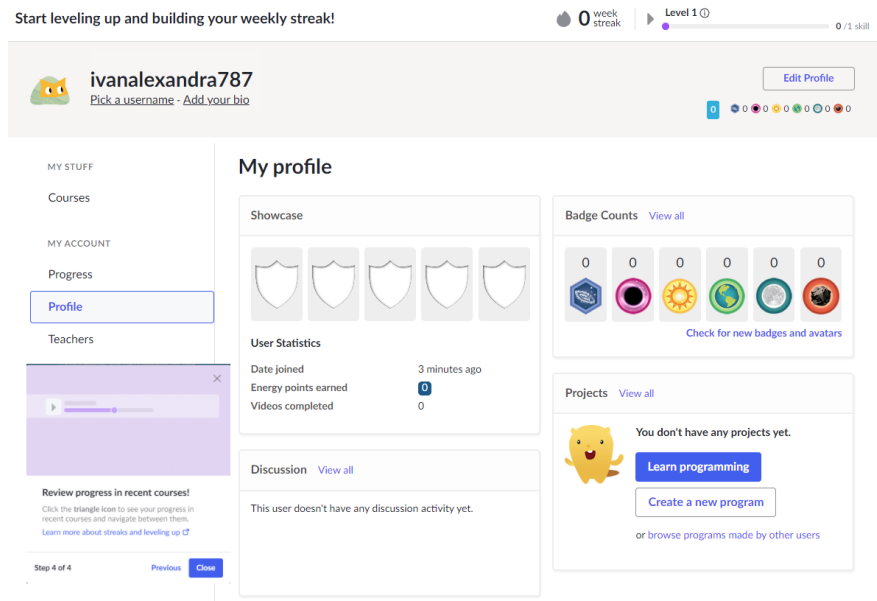


Figura 2.2: Captură din platforma Khan Academy

2.2.3 Coursera

Coursera³ este o platformă de învățare online care pune la dispoziție atât cursuri gratuite, cât și plătite, în colaborare cu universități de prestigiu și companii renumite, utilizând algoritmi de inteligență artificială pentru a adapta experiența educațională, a recomanda cursuri relevante și a urmări progresul utilizatorilor. Un element important al acestei platforme este utilizarea analizei datelor și a machine learning pentru predicția performanței utilizatorilor și oferirea suportului personalizat. Printre funcționalitățile sale predictive se numără un tracker de obiective săptămânale, care sprijină menținerea performanței, precum și evaluări automate și feedback personalizat, bazate în funcție de ritmul de învățare și obstacolele întâmpinate. În plus, Coursera are capacitatea de a anticipa atunci când un utilizator riscă să renunțe la un curs, oferind resurse adiționale sau sugestii pentru a-și îmbunătăți rezultatele. Această platformă se concentrează pe educația avansată și dezvoltarea profesională, furnizând cursuri și certificări recunoscute pe plan internațional, fiind ideală pentru cei care doresc să își consolideze cunoștințele și să progreseze în carieră.

³ Platforma Coursera: <https://www.coursera.org/>, accesat la 14.03.2025

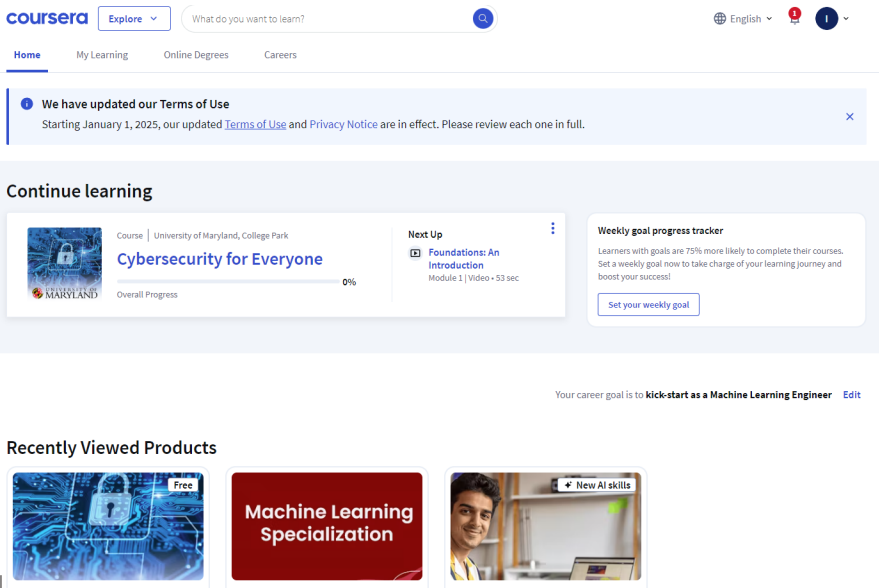


Figura 2.3: Captură din platforma Coursera

2.2.4 Analiză comparativă

Achieve+ reușește să se diferențieze față de platformele cunoscute precum Duolingo, Coursera și Khan Academy prin diverse funcții importante. Toate oferă badges pentru motivarea utilizatorilor și recomandă lecții pentru adaptarea învățării, însă Achieve+ oferă prezicerea ratei de abandon și nota utilizatorilor.

În plus, aceasta pune la dispoziție rapoarte detaliate pentru profesori și statistici despre activitatea studenților, depășind funcțiile limitate ale celorlalte platforme. Drept urmare, profesorii pot monitoriza mai eficient progresul și implicarea studenților.

*	Duolingo	Coursera	Khan Academy	Achieve+
Badges	Da	Da	Da	Da
Recomandarea lectiilor	Nu	Da	Da	Da
Prezicere rata de abandon	Nu	Nu	Nu	Da
Prezicere nota	Nu	Nu	Nu	Da
Rapoarte pentru profesori	Nu	Nu	Partial	Da
Statistici activitate	Nu	Partial	Partial	Da

Figura 2.4: Tabel comparativ

Capitolul 3

Tehnologii folosite

În acest capitol sunt prezentate principalele tehnologii utilizate pentru dezvoltarea aplicației. Scopul acestora a fost să asigure o soluție modernă, scalabilă și ușor de întreținut, care să corespundă cerințelor funcționale și nefuncționale ale aplicației. Se vor detalia platformele și limbajele de programare, framework-urile, baza de date, mediul de dezvoltare și algoritmi implementați.

3.1 Mediul de dezvoltare integrat - Visual Studio

Visual Studio Code reprezintă un editor de cod sursă dezvoltat de Microsoft, dobândind rapid popularitate printre dezvoltatorii web deoarece este prompt, ușor de utilizat și disponibil pe platforme precum Windows, macOS și Linux. Unul dintre principalele avantaje ale acestui editor este suportul pentru IntelliSense, care oferă sugestii contextuale pentru cod, ușurând astfel scrierea acestuia. În plus, Visual Studio Code are integrate instrumente pentru refactorizarea codului, ajutând la optimizare și organizare.[8]

Alegerea acestui editor pentru dezvoltarea aplicației este justificată de diverse avantaje funcționale și practice. Se beneficiază de o gamă largă de extensii, care permit adaptarea mediului de lucru și creșterea funcționalităților, conform cerințelor proiectului. Acesta prezintă și suport integrat pentru debugging, care permite execuția și depanarea codului în multiple limbaje de programare, inclusiv pentru aplicații construite pe platforma .NET Core. Aceste aspecte denotă faptul că Visual Studio Code a fost cea mai bună opțiune pentru construirea aplicației prezentată în această lucrare.

3.2 Platforma și limbajele de programare

Platforma și limbajele de programare reprezintă cheia oricărui proiect software, oferind mijloacele necesare pentru a transforma ideile și cerințele într-o aplicație utilă și funcțională.

Acest subcapitol prezintă limbajele și platforma utilizate în aplicația Achieve+, fiind instrumente care au contribuit la realizarea unei soluții performante.

3.2.1 .NET 8

.NET 8 constituie una dintre versiunile platformei .NET, fiind prezentat cu suport pe termen lung (LTS) și numeroase îmbunătățiri pentru performanță. Este construit alături de C# 12 și oferă posibilitatea creării aplicațiilor cross-platform: web, desktop, mobile sau cloud, fiind compatibil cu Windows, Linux și macOS. De asemenea, asigură compatibilitate cu Unity, integrare cu containere Docker și suport pentru învățarea automată prin ML.NET. În ciuda faptului că .NET 9 este cea mai nouă versiune, .NET 8 se păstrează a fi alegerea recomandată pentru proiectele care necesită o bază solidă.[9]

3.2.2 C#

C# (C Sharp) este un limbaj de programare orientat pe obiecte și tipizat static, dezvoltat de Microsoft, îmbinând performanța, productivitatea și siguranța. Este inspirat de limbaje precum C++ și Java și se deosebește printr-o sintaxă clară, ușor de utilizat și învățat. Limbajul se încadrează în standardele internaționale (ECMA-334, ISO/IEC 23270), fiind profund integrat în ecosistemul .NET. C# este utilizat frecvent pentru dezvoltarea de aplicații desktop, web și mobile.[10]

3.2.3 HTML

Limbajul principal utilizat pentru organizarea și structurarea conținutului paginilor web este **HTML** (HyperText Markup Language). Acesta permite browserului să afișeze diverse elemente, precum texte, imagini sau link-uri, funcționând ca un sistem de marcare. Spre deosebire de CSS, care se ocupă de aspectul vizual, HTML se concentrează pe semantica și logica informației, definind componente precum titluri, paragrafe și butoane, fără a altera aspectul vizual. Fiind derivat din SGML (Standard Generalized Markup Language), este fundamental pentru construirea paginilor web.[11]

3.2.4 CSS

Începând cu anul 1996, **Cascading Style Sheets** (CSS) a revoluționat lumea designului web, specialiștii controlând mai bine aspectul vizual al paginilor. Alături de HTML, care descrie structura paginii, CSS stabilește stiluri precum culori, dimensiuni, aliniere și poziționare. CSS se distinge prin permiterea aplicării concomitente a mai multor stiluri, ușurând personalizarea paginilor în funcție de preferințe sau nevoi. Acest limbaj de stilizare este simplu și

intuitiv, contribuind la îmbunătățirea interfețelor web. Fiind un instrument esențial în dezvoltarea web modernă, CSS permite layout-uri complicate și design-uri adaptabile.[12]

3.2.5 JavaScript

JavaScript reprezintă unul dintre limbajele principale ale web-ului, fiind extrem de folosit în majoritatea site-urilor. Acesta se numără printre cele mai utilizate limbaje din lume și este interpretat de toate browserele desktop, mobile și console de jocuri. Împreună cu HTML și CSS, formează ”triada” fundamentală a dezvoltării web, JavaScript oferind paginilor comportamente dinamice. În ciuda asemănării superficiale cu limbajul Java, a pornit de la un limbaj de scripting simplu și a progresat într-un instrument robust și capabil să susțină aplicații complexe.[13]

3.2.6 Python și Flask

Python este un limbaj de programare de nivel înalt, cu o sintaxă clară și o gamă largă de biblioteci specializate. Acesta reprezintă cea mai bună alegere pentru domenii precum analiza datelor, învățarea automată și inteligență artificială, având biblioteci ca pandas și scikit-learn care permit implementarea modelelor predictive.[14]

Flask este un microframework Python, care a fost utilizat pentru crearea unui server dedicat rulării predicțiilor în cadrul aplicației. Este un framework mic însă este ușor de înțeles și de parcurs, fiind foarte versatil în dezvoltarea diferitelor platforme.[15]

3.3 Arhitectură și framework-uri

Subcapitolul acesta este dedicat arhitecturii și framework-urilor folosite, stând la baza aplicației Achieve+. Este analizată arhitectura Model-View-Controller și este evidențiat avantajul utilizării ASP.NET Core, Entity Framework Core și Bootstrap, care contribuie la dezvoltarea unei aplicații performante.

3.3.1 Arhitectura MVC

Arhitectura Model-View-Controller (MVC) este unul dintre cele mai utilizate design patterns în dezvoltarea aplicațiilor web, întrucât are abilitatea de a separa responsabilitățile în cadrul aplicației. Apărută încă din anii 1970, această arhitectură reușește să fie relevantă și în prezent, fiind utilizată în tehnologii precum ASP.NET Core. Aplicația este împărțită în trei părți distincte, fiecare având scopul său: **Modelul** se ocupă de interacțiunea cu baza

de date și de gestionarea datelor, **View-ul** afișează informațiile utilizatorului și **Controller-ul** reprezintă puntea dintre cele două, răspunzând la acțiunile utilizatorului și administrează interacțiunile dintre componente.[16]

Folosirea arhitecturii MVC pentru o platformă de învățare online, asemenea aplicației Achieve+, reprezintă o alegere inspirată. Spre exemplu, în partea de Model sunt definite toate entitățile importante, precum Lecții, Module, Profil, alături de constrângerile și relațiile dintre ele. Partea de View conține interfețele grafice, pagini pe care le navighează utilizatorul, parcurgând lecții sau vizualizându-și progresul. Controller-ul manevrează interacțiunile, atunci când un utilizator trimite răspunsurile la un test sau accesează o temă, acestea sunt procesate de Controller, apelând Modelul și transmitând rezultatul către View.

Un avantaj al acestei arhitecturi îl constituie organizarea codului și dezvoltarea eficientă. Se poate lucra simultan: funcționalitățile din Controller pot fi modificate și verificate, fără să afecteze celelalte părți ale codului, aplicația fiind mai ușor de întreținut și de extins.

În cadrul aplicației mele, fluxul MVC funcționează în acest mod: atunci când un utilizator solicită un test, sau o lecție, cererea este direcționată către Controller-ul relevant. Acesta colectează datele prin intermediul Modelului, le modifică dacă este nevoie și le trimite către View pentru afișare. În urma trimerii răspunsurilor de către utilizator, acestea sunt preluate din nou de Controller, Modelul le validează și urmează un răspuns în funcție de rezultat. Modul acesta de organizare păstrează o logică clară și o experiență coerentă pentru utilizator.

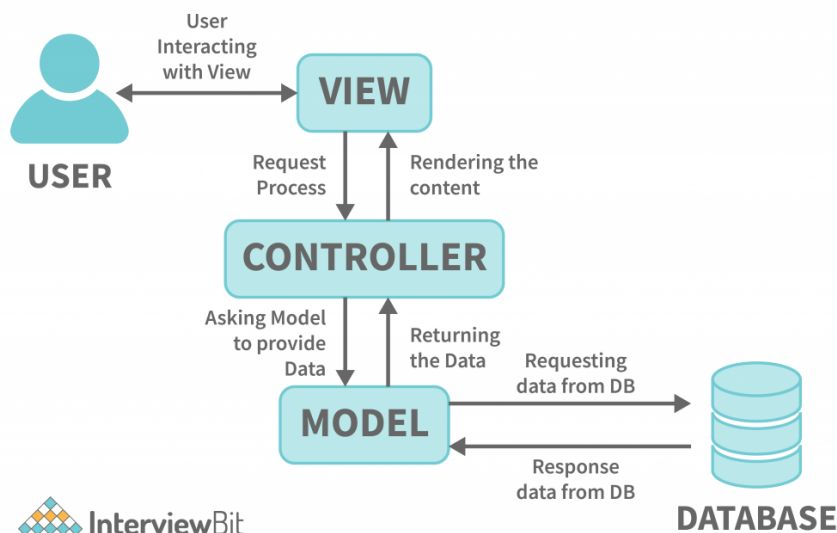


Figura 3.1: Cum funcționează MVC: de la cererea utilizatorului la răspunsul aplicației

Sursa imaginii: <https://www.interviewbit.com/blog/mvc-architecture/>

3.3.2 ASP.NET Core

ASP.NET Core 8 este dezvoltat pe platforma .NET 8 și reprezintă un framework modern, open-source și cross-platform, fiind proiectat pentru crearea aplicațiilor web robuste și scalabile. Acest framework oferă o arhitectură simplă și o gamă largă de instrumente și funcționalități, reducând cantitatea de cod redundant. Acesta detectează cererile HTTP și le reorientează spre componente specializate, precum pagini Razor sau controlere API, care se ocupă de execuția logică a aplicației. Logica principală se poate integra cu infrastructura ASP.NET Core și poate avea acces la baze de date, API-uri externe sau servicii adiționale. Această abordare performantă și modulară le permite dezvoltatorilor să creeze API-uri și aplicații moderne.[17]

3.3.3 Entity Framework Core

Fiind o bibliotecă open-source dezvoltată de Microsoft, Entity Framework Core (EF Core) pune la dispoziție dezvoltatorilor o metodă simplă și intuitivă de a interacționa cu bazele de date. Aceasta funcționează ca un object-relational mapper (ORM), corelând structura relațională a bazei de date și modelul orientat pe obiecte utilizat în codul aplicației. Acest aspect permite manevrarea entităților și obiectelor fără a scrie în mod direct interogări SQL. Folosind această bibliotecă, se accelerează procesul de dezvoltare, în mod deosebit pentru programatorii obișnuiți cu limbajele de programare. [18]

3.3.4 Bootstrap

Bootstrap reprezintă un framework front-end extrem de popular, care este utilizat pentru crearea rapidă de site-uri și aplicații web responsive. Acesta folosește HTML, CSS și JavaScript pentru a pune la dispoziție o gamă bogată de componente predefinite, precum tabele, butoane, ușurând crearea unor interfețe moderne și funcționale. Dezvoltatorii pot integra ușor funcționalități interactive, fără a scrie cod de la zero. Acest framework este o opțiune practică pentru proiecte simple, dar și pentru aplicații web de dimensiuni mari, oferind un balans între viteza de dezvoltare și un rezultat calitativ, fiind flexibil și adaptabil la diferite dispozitive.[19]

3.4 Baza de date

Acest subcapitol este dedicat prezentării bazei de date și este explicată platforma de gestionare a bazelor de date și a limbajului folosit.

3.4.1 Microsoft SQL Server

Microsoft SQL Server este o platformă de gestionare a bazelor de date relaționale, fiind recunoscută pentru performanța sa, protecția datelor și integrarea cu ecosistemul Microsoft. O dată cu versiunea 2014, a integrat tehnologii in-memory pentru OLTP (Online Transaction Processing) și data warehousing, optimizând viteza de procesare a tranzacțiilor și a interogărilor. În plus, a furnizat suport extins pentru medii hibride și a ușurat trecerea către cloud și integrarea cu servicii precum Azure. Microsoft SQL Server garantează o infrastructură sigură și scalabilă.[20]

3.4.2 SQL

Apărut în anii 1970, SQL (Structured Query Language) este limbajul standard pentru gestionarea și manipularea bazelor de date relaționale. În ciuda faptului că este un limbaj mai vechi, SQL continuă să domine în tehnologia modernă datorită flexibilității și eficienței sale în lucrul cu volume mari de date. Acest limbaj se diferențiază de alte sisteme de gestionare prin capacitatea sa de a manevra operațiuni la scară largă, cât și interogări asupra datelor individuale. Deoarece este un limbaj versatil și extrem de performant, încercările de a-l înlocui au eșuat în mod regulat. Astfel, SQL rămâne un instrument fundamental în dezvoltarea de software.[21]

3.5 Algoritmi și metode folosite

În dezvoltarea unui sistem software, alegerea și implementarea corectă a algoritmilor este crucială, întrucât afectează în mod direct performanța, adaptabilitatea și securitatea sistemului. În această secțiune sunt evidențiați diferiți algoritmi utilizați în aplicație, de la metode de securizare a datelor până la algoritmi de recomandări și predicție.

3.5.1 BCrypt

Dezvoltat în 1999 de către Niels Provos și David Mazieres, **BCrypt** reprezintă un algoritm de hashing conceput pentru stocarea în siguranță a parolelor. Bazat pe cifrul Blowfish, algoritmul se face remarcant prin abilitatea sa de a se adapta la creșterea puterii de calcul, oferind protecție împotriva atacurilor brute-force. Acesta oferă posibilitatea de a ajusta complexitatea procesului de hashing printr-un parametru de cost, distingându-se de algoritmi mai vechi, precum DES-crypt sau MD5-crypt, care prezintă vulnerabilități odată cu îmbunătățirea hardware-ului. Parametrul de cost controlează numărul de iterații ale funcției de hashing, asigurând faptul că generarea unui hash rămâne destul de lentă pentru descurajarea atacatorilor, inclusiv pe calculatoarele moderne.[22]

În implementarea aplicației, am folosit BCrypt.NET, o bibliotecă pentru platforma .NET, care îmbină toate avantajele algoritmului BCrypt. Acesta asigură siguranța parolelor stocate, oferind protecție împotriva atacurilor offline. Alegerea acestui algoritm furnizează un grad ridicat de securitate pe termen lung, fiind adaptat la creșterea capacității de procesare a sistemelor noi.

3.5.2 Algoritmul Cosine Similarity

Cosine Similarity măsoară gradul de asemănare dintre doi vectori într-un spațiu multidimensional și reprezintă o tehnică esențială pentru sistemele de recomandare. Potrivit formulei matematice[23], calculul se bazează pe cosinusul unghiului dintre doi vectori:

$$\text{cosine_similarity}(A, B) = \frac{\langle A, B \rangle}{\|A\| \cdot \|B\|}$$

Rezultatul funcției variază de la 0 la 1: unde 0 reprezintă nicio similitudine, iar 1 reprezintă vectori identici. Acest algoritm este optim pentru datele în care proporționalitatea este mai importantă decât valoarea absolută, întrucât se concentrează pe orientarea vectorilor. [24]

Se folosește în sistemele de recomandare pentru găsirea elementelor similare pe baza profilului utilizatorului sau particularităților conținutului. Spre exemplu, un utilizator poate fi exprimat printr-un vector al intereselor, dezvoltat pe interacțiuni sau preferințe, iar fiecare lecție poate fi exprimată printr-un vector binar potrivit unor taguri asociate. Prin compararea acestor vectori, sistemul poate recomanda conținut personalizat, determinând gradul de similitudine dintre lecțiile disponibile și interesele utilizatorului.

În aplicația Achieve+, algoritmul Cosine Similarity a fost utilizat pentru generarea recomandărilor personalizate de lecții. Fiecare utilizator are asociat câte un vector care exprimă frecvența interacțiunilor cu diverse taguri (precum matematică, gramatică, limbi străine etc.), iar lecțiile neparcuse au fost exprimate ca vectori binari, depinzând de tagurile asociate acestora. În urma normalizării vectorilor, scorul de similaritate pentru fiecare lecție în raport cu profilul utilizatorului a fost calculat, lecțiile cu cele mai mari valori fiind afișate. Procesul acesta a condus spre crearea unui sistem de recomandări riguros, care se poate adapta la comportamentul fiecărui utilizator.

Acest algoritm a fost ales întrucât este eficient în analiza datelor de dimensiuni mari și generează rezultate stabile, indiferent de scala vectorilor. Este potrivit pentru conținutul divers și dispersat al platformelor educaționale, reușind să furnizeze recomandări personalizate și o experiență academică eficientă.

3.5.3 Algoritmul de predicție

Model	R^2	MAE	MSE	RMSE	SSE
Random Forest	0.922	1.78	8.25	2.87	808.99
Decision Tree	0.783	2.97	22.91	4.79	2245.00
Linear Regression	0.999	0.26	0.10	0.32	9.85

Tabelul 3.1: Compararea modelelor de regresie pe baza metricilor de performanță

Tabelul de mai sus prezintă o comparație a modelelor de regresie testate, Random Forest, Decision Tree și Linear Regression, folosind mai mulți indici de performanță: coeficientul de determinare R^2 , eroarea medie absolută (MAE), eroarea medie pătratică (MSE), rădăcina erorii medii pătratice (RMSE) și suma erorilor pătratice (SSE).[23]

Modelul Linear Regression a obținut cele mai bune performanțe conform tuturor indicilor de performanță. Având cel mai mare coeficient $R^2 = 0.999$, se indică faptul că acesta explică foarte bine variația datelor. În plus, are cele mai mici valori pentru erori, sugerând ca modelul de regresie liniară este precis pentru acest set de date.

Pe locul al doilea, ca performanță, se află modelul Random Forest, cu un coeficient de determinare $R^2 = 0.922$. Deși coeficientul a avut o valoare bună, valorile erorilor sunt mai mari decât cele ale regresiei liniare, însă se dovedește a fi semnificativ mai performant decât Decision Tree.

În urma analizei comparative, am ales să folosesc modelul de regresie liniară, fiind cel mai potrivit pentru setul de date analizat.

3.5.3.1 Metodologie

Aplicația Achieve+ utilizează două seturi de date distincte¹ pentru a analiza și prezice performanța academică (rata de abandon și nota finală). Datele au fost obținute de la o universitate nord-americană, în urma unui curs științific oferit într-un format hibrid, fiind înscriși 486 de studenți.

Primul dataset reprezintă nivelul de implicare al studenților, preluat dintr-un jurnal de evenimente minuțios colectat de la sistemul de management al învățării (LMS) al universității, denumit OWL. Acest set de date conține indicatori de interacțiune și efort, precum numărul de accesări al site-ului, numărul de accesări al materialelor de curs, durata trimiterii temelor etc. Determinat prin clustering K-means, datasetul are drept coloană finală nivelul de implicare, care poate fi "Ridicat" (H) sau "Scăzut" (L). [25]

Al doilea dataset are ca scop performanța academică a studenților, fiind compus din notele acestora la diverse tipuri de evaluări: teste, teme și examene. Un aspect important al

¹ Am preluat ambele dataseturi de aici: <https://github.com/Western-OC2-Lab/Student-Performance-and-Engagement-Prediction-eLearning-datasets>

acestui dataset este "Student Category", care împarte studenții în două categorii nominale: "Bun" (G) pentru cei cu o notă peste 60 și "Slab" (W) pentru restul, aceștia având șansa de a nu promova cursul.[26]

3.5.3.2 Random Forest

Random Forest este un algoritm de învățare automată bazat pe mai mulți arbori de decizie, fiind dezvoltat pentru obținerea unor predicții mai bune. Fiecare arbore este antrenat pe un eșantion aleatoriu de date și caracteristici, astfel corelația dintre arbori este redusă și predicția finală (realizată prin vot majoritar sau medie) este mai precisă.[27] Inițial, acesta fusese algoritmul ales pentru predicția notei finale, însă în urma testărilor, regresia liniară a oferit o performanță superioară și a fost preferată în final.

3.5.3.3 Arbori de decizie

Un arbore de decizie este o structură ierarhică unde fiecare nod corespunde unei decizii, iar frunzele corespund unui rezultat sau unei etichete de clasă. În cadrul fiecărui nod intern sunt evaluate una sau mai multe valori ale unor atribute și în funcție de rezultat sunt urmate două sau mai multe ramuri. Fiecare ramură reprezintă o variantă posibilă a deciziei luate în acel punct. Aceste ramuri sunt separate între ele și acoperă toate opțiunile posibile, orice combinație de valori ducând la o ramură.[28] Am ales să nu utilizez arbori de decizie întrucât performanța acestui algoritm a fost semnificativ mai slabă în comparație cu ceilalți algoritmi testați.

3.5.3.4 Modele liniare

Regresia liniară este un model matematic unde iese în evidență relația dintre o variabilă dependentă și una sau mai multe variabile independente, sub presupunerea ca relația este de tip liniar. Acest model este adeseori utilizat în predicții, iar pentru dataset-ul folosit a avut cea mai bună performanță. [29]

Regresia logistică este un model folosit pentru prezicerea unui rezultat binar, adică într-o situație în care răspunsul poate fi doar "da" sau "nu" ("1" sau "0"). Spre deosebire de regresia liniară, aceasta transformă rezultatul într-o probabilitate, utilizând o funcție sigmoidă. În cadrul aplicației, regresia logistică a fost folosită pentru estimarea riscului de abandon. [30]

Capitolul 4

Proiectarea și dezvoltarea aplicației Achieve+

În acest capitol voi prezenta modul în care aplicația Achieve+ a fost dezvoltată, parcurgând etapele esențiale, începând cu analiza cerințelor funcționale și nefuncționale, realizată prin cercetarea literaturii de specialitate și a aplicațiilor similare din domeniul de E-Learning. Pe baza informațiilor extrase, au fost stabilite nevoile cheie ale utilizatorilor, precum accesul facil, adaptabilitatea conținutului și furnizarea feedback-ului personalizat. Ulterior, au fost elaborate cazurile de utilizare, reprezentând principalele interacțiuni între utilizator și sistem, iar mai apoi, au fost realizate diagramele de modelare. Se începe cu autentificarea pe baza unui rol (elev, profesor), care determină accesul la funcționalități precum gestionarea lecțiilor, monitorizarea progresului sau predicția unui rezultat.

4.1 Cerințe funcționale

Tabelul de mai jos prezintă cerințele funcționale ale aplicației pentru o selecție reprezentativă de cazuri de utilizare:

F1. Adăugare lecție	F1.1 Afișează un formular pentru datele noilor lecții
	F1.2 Verifică datele primite
	F1.3 Memorează lecția pe suport persistent
	F1.4 Afișează un mesaj de confirmare a operației
	F1.5 Dacă datele sunt incorecte sau incomplete, afișează un mesaj de eroare

F2. Parcurgere lecție	F2.1 Afișează conținutul lecției
	F2.2 Memorează progresul pe suport persistent
	F2.3 Actualizează progresul
	F2.4 Confirmă finalizarea lecției
F3. Administrare test	F3.1 Afișează lista testelor existente și opțiunile disponibile (adaugă, editează, șterge)
	F3.2 Afișează formularul pentru introducerea datelor testului
	F3.3 Verifică datele primite
	3.4 Salvează testul pe suport persistent
	3.5 Afișează un mesaj de confirmare
	3.6 Dacă datele sunt incorecte sau incomplete, afișează un mesaj de eroare
F4. Câștigare badges	F4.1 Verifică dacă sunt îndeplinite condițiile pentru badge
	F4.2 Salvează badge-ul pe suport persistent
	F4.3 Afișează badge-ul obținut în pagina "Progres"

4.2 Cerințe nefuncționale

Cerințele nefuncționale [31], sau atributele de calitate, reprezintă constrângeri pe care un sistem software trebuie să le îndeplinească.[32] Acestea explică modul în care sistemul ar trebui să opereze, incorporând aspecte precum ușurința utilizării sistemului, robustețea, scalabilitatea, securitatea și suportul persistent. Cerințele nefuncționale sunt cruciale pentru garantarea unei experiențe eficiente, sigure și coerente pentru utilizatori, alături de funcționarea stabilă a sistemului în diferite scenarii.

Ușurința utilizării sistemului reprezintă una dintre cerințele nefuncționale esențiale, denotând intenția de a proiecta un sistem ușor de utilizat, indiferent de nivelul utilizatorilor de cunoștințe tehnice. Interfața grafică va fi simplă și intuitivă, facilitând o interacțiune accesibilă.

Pentru îmbunătățirea **robusteții**, sistemul va avea integrate mecanisme de validare a datelor introduse, pentru prevenirea stocării și procesării datelor incomplete sau greșite. Sistemul va asigura mesaje de eroare clare, care vor veni în ajutorul utilizatorului pentru remedierea problemelor. Acest aspect contribuie la menținerea consistenței și integrității datelor.

În ceea ce privește **suportul persistent**, sistemul va utiliza un sistem de gestiune a bazelor de date (SGBD) performant și robust, garantând stocarea în siguranță și integritatea datelor relevante, precum informații despre utilizatori, lecții, module de învățare și progresul individual al fiecărui utilizator. În plus, datele vor putea fi accesate în timp real, sistemul având integrate măsuri de redundanță și backup periodic, garantând faptul că datele nu vor fi pierdute sau deteriorate.

Scalabilitatea reprezintă o altă cerință nefuncțională esențială, în care arhitectura sistemului va putea susține o creștere progresivă a numărului de utilizatori și a volumului de date procesate. Soluțiile implementate vor asigura menținerea performanțelor operaționale, chiar și în condițiile de utilizare intensivă.

Referitor la **securitate**, sistemul va integra măsuri tehnice pentru protejarea integrității, confidențialității și disponibilității datelor. Elementele esențiale vor fi criptarea datelor sensibile, autentificarea pe baza unor credențiale sigure și includerea unor politici de control al accesului. Mai mult, un mecanism automat de salvare a datelor asigură recuperarea acestora în cazul unor probleme neprevăzute.

4.3 Definirea cazurilor de utilizare

Cazurile de utilizare software reprezintă o ”unitate coerentă de funcționalitate oferită de sistem” și evidențiază felul în care acesta interacționează cu actorii pentru atingerea unui anumit scop.[32] Am ales să descriu patru cazuri de utilizare reprezentative pentru aplicația Achieve+.

4.3.1 Cazul de utilizare Adăugare Lecție

Acest caz descrie comportamentul sistemului și interacțiunea dintre profesor și sistem pentru adăugarea unei lecții noi. Actorul software este profesorul, iar evenimentul este declanșat de cererea acestuia de adăugare a unei lecții noi. Profesorul trebuie să fie autentificat și să apese butonul de ”Creare lecție”, urmând ca la final sistemul să înregistreze lecția pe suport persistent. Referințele încrucișate sunt F1.1 - F1.5. Voi prezenta fluxul principal sub forma unui tabel.

Profesor	Sistem
1. Cere să adauge o lecție	2. Afișează un formular pentru datele noii lecții
3. Completează formularul și trimite datele [A1]	4. Verifică datele primite [A2]
	5. Memorează lecția pe suport persistent
	6. Afișează un mesaj de confirmare a operației

Fluxuri alternative:**[A1]: Profesorul renunță**

1. Profesorul renunță să adauge o lecție
2. Fluxul se termină

[A2]: Date incorecte/incomplete

1. Sistemul afișează un mesaj de eroare
2. Fluxul continuă cu pasul 2 din fluxul principal

4.3.2 Cazul de utilizare Parcurgere Lecție

Se descrie comportamentul sistemului și interacțiunea dintre student și sistem pentru parcurgerea unei lecții. Actorul software este, de data aceasta, studentul, care accesează o lecție. Studentul trebuie să fie autentificat și să fi apăsat pe butonul de ”Începe lecția” al unei lecții. Ca urmare a acestui caz, sistemul a înregistrat progresul studentului pe suport persistent. Referințele încrucișate sunt F2.1-F2.4.

Student	Sistem
1. Accesează o lecție	2. Afișează conținutul lecției
3. Parcurge conținutul	4. Memorează progresul pe suport persistent
	5. Actualizează progresul
6. Finalizează lecția	7. Confirmă finalizarea lecției

4.3.3 Cazul de utilizare Administrare Test

Descrie comportamentul și interacțiunea dintre profesor și sistem pentru adăugarea, modificarea sau ștergerea unui test asociat unei lecții. Profesorul este actorul software și apasă pe butonul aferent acțiunii pe care dorește să o facă. Acesta este autentificat și este singurul care are drepturi de editare. La final, testul este adăugat, modificat sau șters, iar modificările sunt salvate pe suport persistent. Referințele încrucișate sunt F3.1 - F3.6

Profesor	Sistem
1. Accesează secțiunea testelor	2. Afișează lista testelor existente și opțiunile disponibile (adaugă, editează, șterge)
3. Alege să adauge un test nou	4. Afișează formularul pentru introducerea datelor testului
5. Completează și trimite formularul [A2]	6. Verifică validitatea datelor [A1]
	7. Salvează testul pe suport persistent
	8. Afișează un mesaj de confirmare

Fluxuri alternative:**A1]: Date incomplete/incorecte**

1. Sistemul afișează un mesaj de eroare
2. Fluxul revine la pasul 4 din fluxul principal

[A2]: Profesorul renunță

1. Profesorul anulează acțiunea
2. Fluxul se încheie fără a salva modificări

4.3.4 Cazul de utilizare Câștigare badges

Describe comportamentul sistemului și interacțiunea dintre sistem și student pentru obținerea unor badges ca recompensă în urma finalizării anumitor activități. Actorul software este studentul și finalizează o activitate pentru care este prevăzută recompensarea cu un badge. Studentul trebuie să fie autentificat și să fi îndeplinit condițiile pentru obținerea unui badge. Ca urmare, sistemul atribuie un badge și îl salvează în pagina de progres a studentului. Referințele încrucișate sunt F4. - F4.

Student	Sistem
1. Finalizează o activitate (lecție, test, temă)	2. Verifică dacă sunt îndeplinite condițiile pentru badge
	4. Salvează badge-ul pe suport persistent
5. Primește notificare	6. Afișează badge-ul obținut în pagina "Progres"

4.4 Diagramele de modelare

În acest subcapitol sunt ilustrate diagramele de modelare pentru cazurile de utilizare specifice aplicației Achieve+.

4.4.1 Diagrama cazurilor de utilizare

Diagrama cazurilor de utilizare descrie contextul general al sistemului, identificând interacțiunile dintre actor și sistem sub forma unor tranzacții. [32]4.1

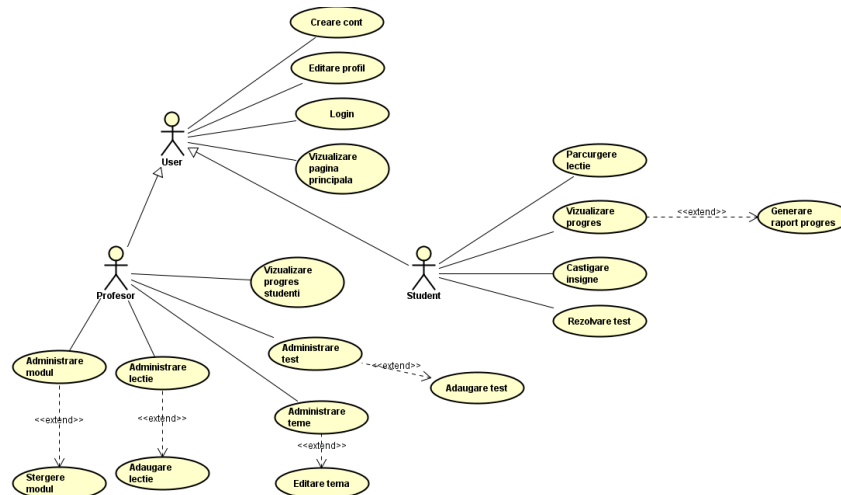
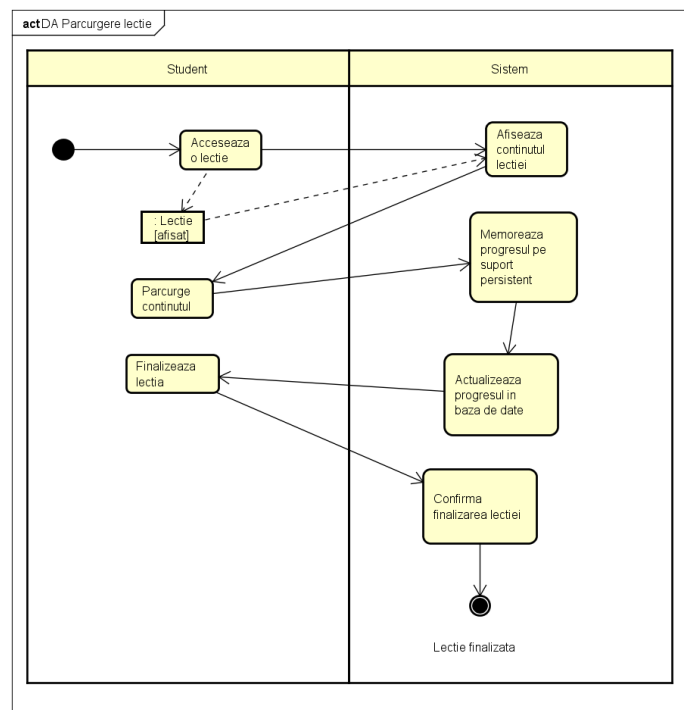


Figura 4.1: Diagrama cazurilor de utilizare

4.4.2 Diagrama de activitate

Diagrama de activitate este un graf de activități folosit pentru a descrie pașii executați dintr-un proces sau workflow. Activitățile sunt conectate prin săgeți care indică ordinea execuției, fiind grupate în "swimlanes" care indică cine le execută. [32]4.2, 4.3



powered by Astah

Figura 4.2: Diagrama de activitate - Parcurgere lectie

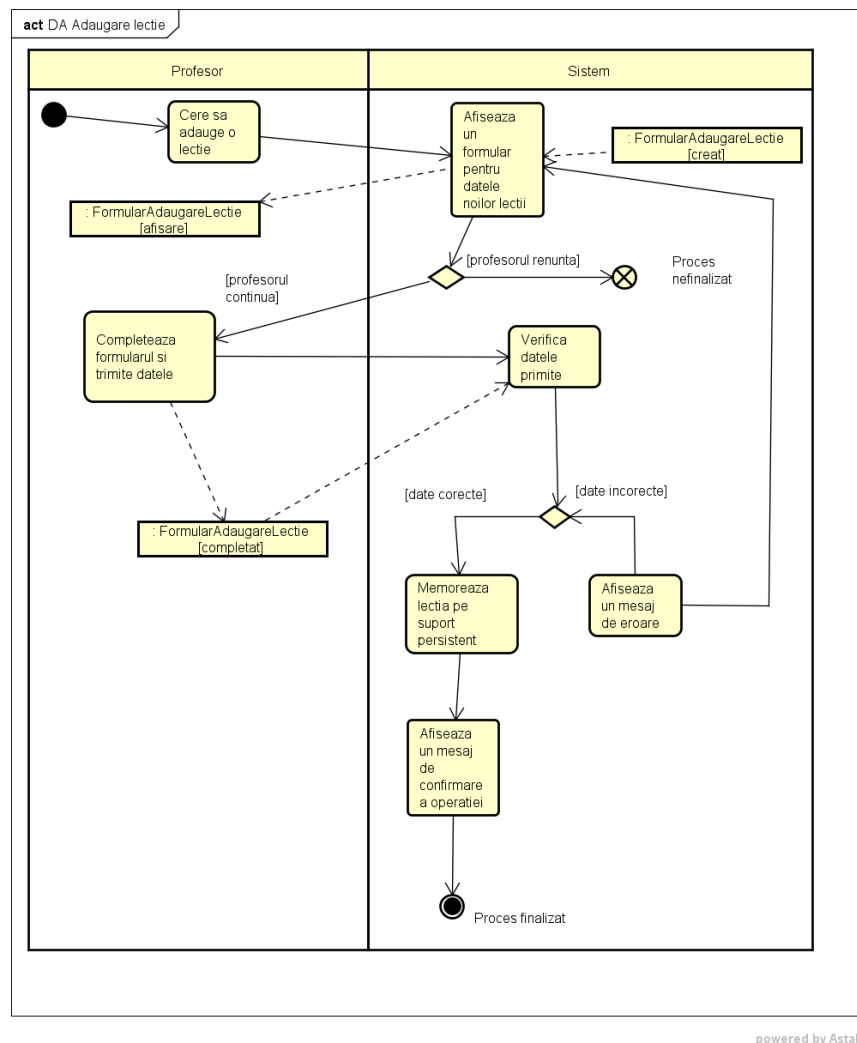


Figura 4.3: Diagrama de activitate - Adăugare lecție

4.4.3 Diagrama de secvențe

Diagramele de secvențe evidențiază interacțiunea dintre actorii software și sistem. [32] 4.4, 4.5

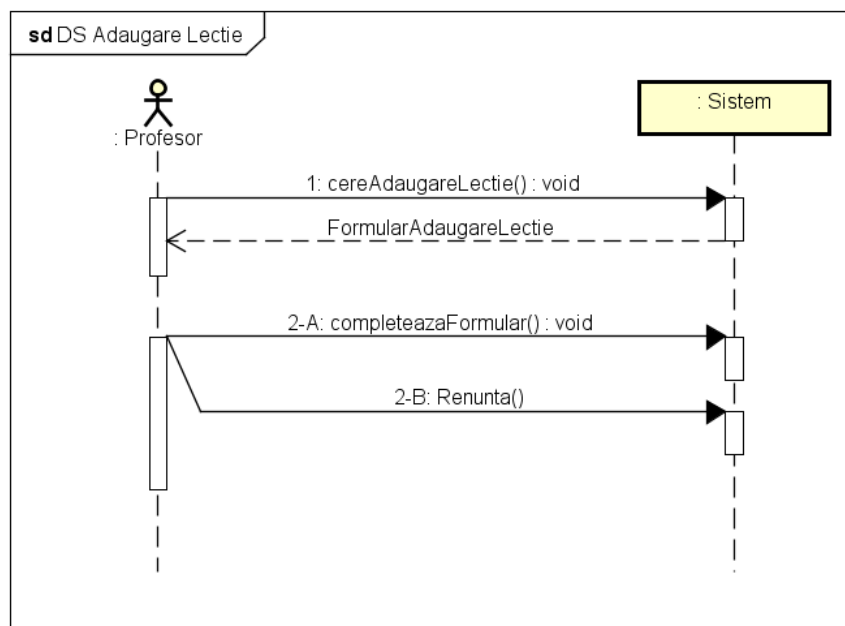


Figura 4.4: Diagrama de secvențe - Adăugare lecție

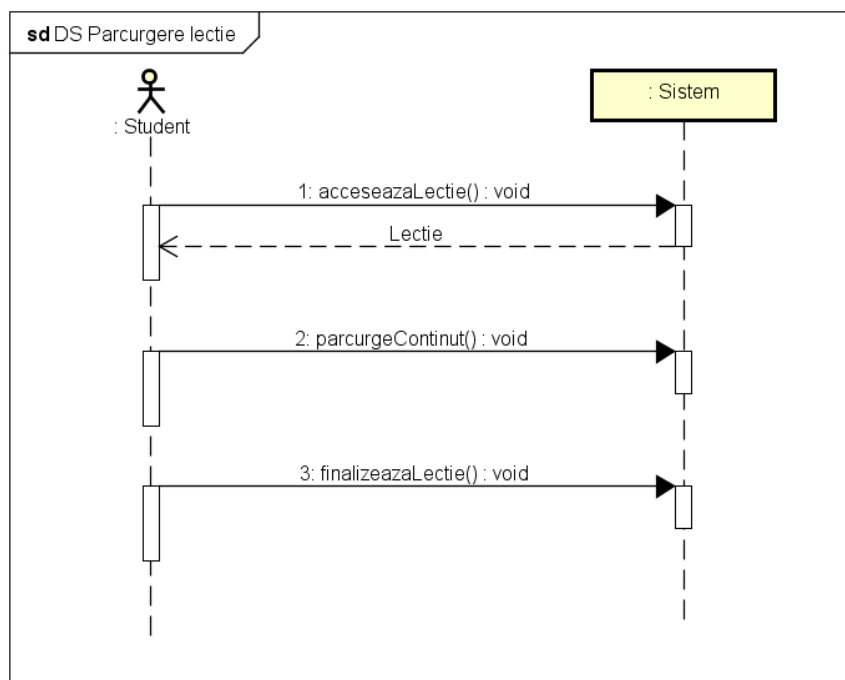


Figura 4.5: Diagrama de secvențe - Parcurgere lecție

4.4.4 Diagrama de secvențe în proiectare

În figurile următoare sunt reprezentate diagramele de secvențe în proiectare, în care am ilustrat interacțiunile componentelor MVC.4.6, 4.7

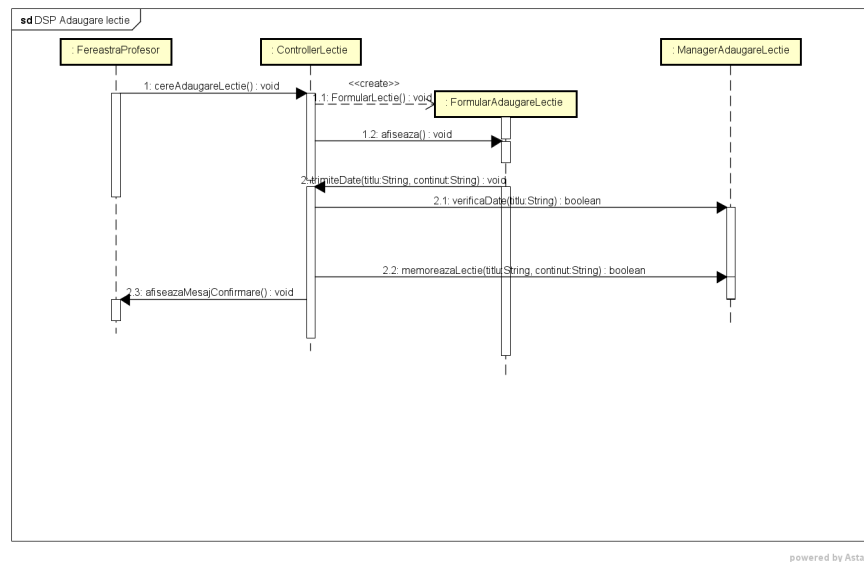


Figura 4.6: Diagrama de secvențe în proiectare - Adăugare lecție

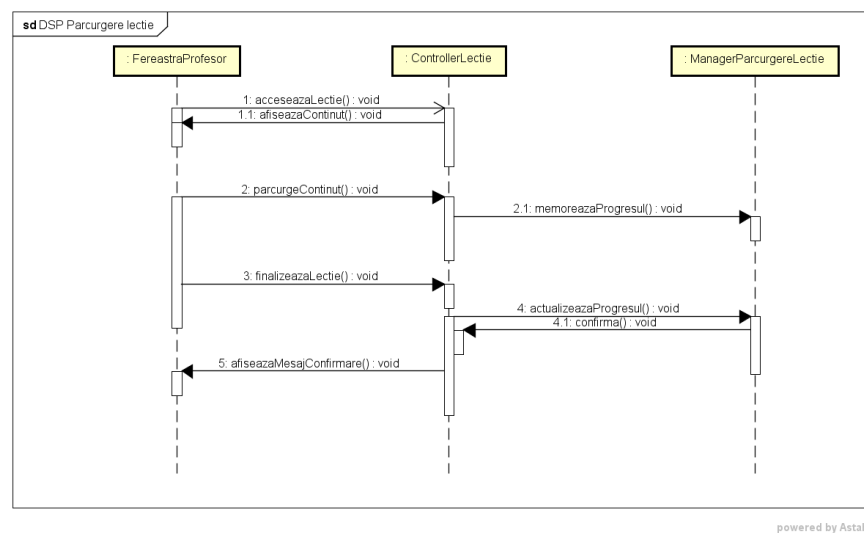


Figura 4.7: Diagrama de secvențe în proiectare - Parcurgere lecție

4.5 Proiectarea bazei de date

O bază de date relațională constituie un model al datelor bazat pe algebra relațională, fiind introdus de către E.F.Codd în 1970. Datele sunt organizate sub formă de tabele, fiecare rând corespunzând unui tuplu (înregistrări), iar coloanele definesc atributele acestora. Relațiile dintre tabele sunt stabilite prin cheile primare (identificatori unici) și prin cheile străine (care fac referire la alte chei primare din alte tabele). Acest model are capacitatea de a realiza interogări complexe, dispunând de limbajul SQL.[33] [34]

Modelul Entitate-Asociere (E-A) este un model de date conceptual, introdus de Peter Chen în 1976 pentru ușurarea proiectării bazelor de date. Acest model este utilizat preponderent în faza de proiectare conceptuală, deoarece descrie datele în mod clar și intuitiv, înainte de implementarea acestora într-un sistem de gestiune a bazelor de date (SGBD). Elementele fundamentale sunt entitățile (obiecte din lumea reală care pot fi identificate în mod distinctiv) și relațiile (asocierile)[34], care reprezintă modul în care entitățile interacționează sau sunt conectate. [35]

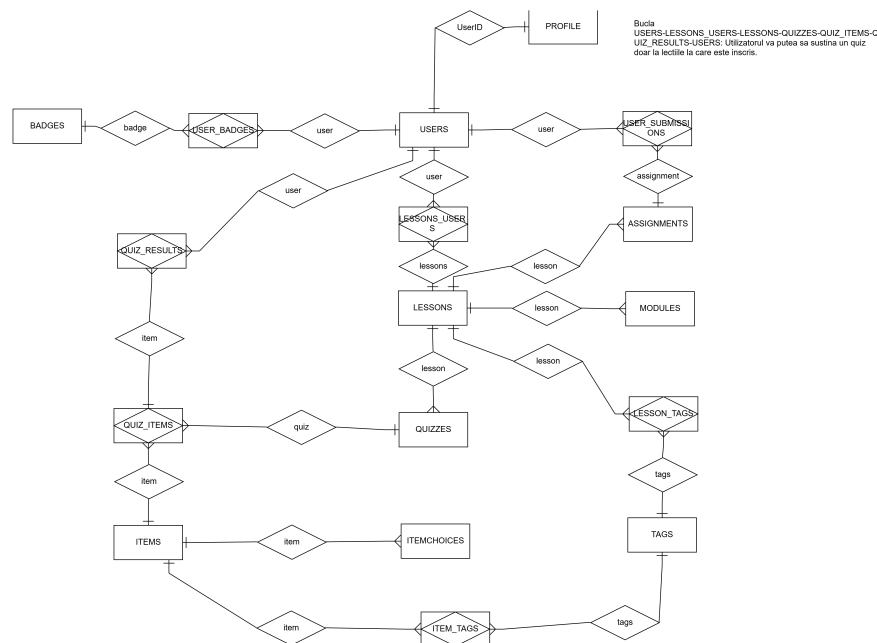


Figura 4.8: Diagrama Entitate-Asociere (ERD) pentru aplicația Achieve+

4.5.1 Restricții

Nu pot exista doi utilizatori cu același nume de utilizator. (R_1)

Nu pot exista doi utilizatori cu același email. (R_2)

Nu pot exista două profile asociate aceluiași nume de utilizator. (R_3) (Justificare relație 1 - 1 între entitatea USERS și entitatea PROFILES)

O trimitere trebuie să fie asociată unui utilizator existent și unei teme existente. (R_4)

O lecție trebuie să fie asociată unui modul existent. (R_5)

Un utilizator va putea să susțină un test doar la lecțiile pe care le-a început sau parcurs. (R_6) Rezolvare buclă: **USERS** → **LESSONS_USERS** → **LESSONS** → **QUIZZES** → **QUIZ_ITEMS** → **QUIZ_RESULTS** → **USERS**

Un test trebuie să fie asociat unei lecții existente. (R_7)

O opțiune de răspuns trebuie să fie asociată unei întrebări existente. (R_8)

Un rezultat trebuie să fie asociat unui utilizator existent, unui test existent și unei întrebări existente. (R_9)

O opțiune de răspuns trebuie să fie asociată unei întrebări. (R_{10})

Un rezultat trebuie să fie asociat unui utilizator existent, unui test existent și unei întrebări existente. (R_{11})

Nu pot exista două etichete cu același nume. (R_{12})

O asociere între lecție și etichetă trebuie să facă referire la lecții și etichete existente. (R_{13})

O asociere între întrebare și etichetă trebuie să facă referire la întrebări și etichete existente. (R_{14})

Nu pot exista două insigne cu același nume. (R_{15})

O insignă acordată trebuie să fie asociată unui utilizator existent și unei insigne existente. (R_{16})

Un utilizator nu poate primi aceeași insignă de mai multe ori. (R_{17}) (Combinatie unică **userID + badgeID**)

Progresul unui utilizator într-o lecție trebuie să fie între 0 și 100. (R_{18})

Un utilizator nu poate avea mai multe înregistrări de progres pentru aceeași lecție. (R_{19}) (Combinatie unică **userID + lessonID**)

Un utilizator poate trimite o singură trimitere la o temă. (R_{20})

O lecție trebuie să aparțină unui modul. (R_{21})

Nu pot exista două întrebări cu același conținut (R_{22})

4.5.2 Descrierea entităților

USERS - reprezintă utilizatorii platformei și stochează informații de bază despre aceștia. Fiecare utilizator are un nume, o parolă criptată, o adresă de mail, un rol (poate fi student sau profesor), data înregistrării și data ultimei autentificări.

PROFILES - extinde informațiile despre utilizatori și adaugă detalii suplimentare, precum numele complet și calea către imaginea de profil.

ASSIGNMENTS - conține temele pe care studenții trebuie să le completeze. Fiecare temă are un titlu, o descriere, o dată de început, o dată limită și un scor maxim.

USER_SUBMISSIONS - această entitate asociativă stochează soluțiile trimise de utilizatori pentru temele asignate. Fiecare soluție conține data trimiterii, nota obținută, calea către fișierul încărcat, comentarii adiționale și un status care indică starea evaluării.

LESSONS - înregistrează lecțiile disponibile în platformă. Fiecare lecție are un titlu, conținutul propriu-zis și data creării.

MODULES - grupează lecțiile în module organizate. Fiecare modul are un nume, o descriere și data creării.

QUIZZES - această entitate conține testele asociate lecțiilor. Fiecare test include un titlu, durata în minute, categoria din care face parte (normal, parțial sau examen), o descriere, durata în minute și data creării.

QUIZ_ITEMS - această entitate asociativă leagă chestionarele de întrebări (items). Fiecare înregistrare conține poziția întrebării în chestionar.

ITEMS - stochează întrebările care pot fi folosite în teste. Fiecare item are conținutul întrebării, un tip (spre exemplu, grilă), un nivel de dificultate și data creării.

ITEMCHOICES - această entitate conține opțiunile de răspuns pentru întrebările de tip grilă. Fiecare opțiune conține textul opțiunii și un indicator care specifică dacă opțiunea este corectă.

QUIZ_RESULTS - înregistrează rezultatele utilizatorilor la chestionare. Fiecare rezultat are un indicator care arată dacă răspunsul a fost corect și data încercării.

LESSONS_USERS - această entitate asociativă urmărește progresul utilizatorilor în lecții. Fiecare înregistrare conține data începerii, data finalizării, progresul, data ultimei accesări, timpul petrecut și numărul de încercări.

TAGS - conține etichetele folosite pentru a categoriza conținutul în aplicație. Fiecare etichetă are un nume.

LESSON_TAGS - această entitate asociativă acordă lecțiilor etichete.

BADGES - stochează insignele care pot fi acordate utilizatorului pentru realizări și motivare. Fiecare insignă are un nume, o descriere, calea către poza și data creării.

USER_BADGES - această entitate asociativă înregistrează insignele acordate utilizatorilor. Fiecare înregistrare conține data acordării.

4.5.3 Modelul MRD

USERS (userID, username, password_hash, email, role, created_at, last_login, LoginCounts)

Chei semantice: username, email

Constrângeri de domeniu:

- ▷ DOM(username) = NVARCHAR(50)
- ▷ DOM(password_hash) = NVARCHAR(255)
- ▷ DOM(email) = NVARCHAR(20) UNIQUE
- ▷ DOM(role) = $\{n \in \text{NVARCHAR} \mid n \in \{\text{'student'}, \text{'profesor'}\}\}$
- ▷ DOM(created_at) = DATETIME DEFAULT GETDATE()
- ▷ DOM(last_login) = DATETIME

Constrângeri de existență: username, password_hash, email

Constrângeri de tuplu: created_at < last_login

PROFILES (profileID, userID, full_name, avatar_url)

Chei semantice: userID

Constrângeri de domeniu:

- ▷ DOM(username) = NVARCHAR(50)
- ▷ DOM(password_hash) = NVARCHAR(255)
- ▷ DOM(email) = NVARCHAR(20) UNIQUE
- ▷ DOM(role) = $\{n \in \text{NVARCHAR} \mid n \in \{\text{'student'}, \text{'profesor'}\}\}$
- ▷ DOM(created_at) = DATETIME DEFAULT GETDATE()
- ▷ DOM(last_login) = DATETIME

Constrângeri de existență: userID

Constrângeri de integritate referențială:

userID $\subset$ USERS.userID

MODULES (moduleID, module_name, module_description, created_at)

Chei semantice: module_name

Constrângeri de domeniu:

- ▷ DOM(module_name) = NVARCHAR(255)
- ▷ DOM(created_at) = DATETIME DEFAULT GETDATE()

Constrângeri de existență: module_name

LESSONS (lessonID, moduleID, lesson_title, lesson_content, created_at)

Chei semantice: moduleID * lesson_title

Constrângeri de domeniu:

- ▷ $\text{DOM}(\text{lesson_title}) = \text{NVARCHAR}(255)$
- ▷ $\text{DOM}(\text{lesson_content}) = \text{NVARCHAR}(\text{MAX})$
- ▷ $\text{DOM}(\text{created_at}) = \text{DATETIME DEFAULT GETDATE}()$

Constrângeri de existență: `moduleID`, `lesson_title`, `created_at`

Constrângeri de integritate referențială: `moduleID` $\subset$ `MODULES.moduleID`

LESSONS_USERS (`lessons_users_ID`, `userID`, `lessonID`, `start_date`, `finish_date`, `progress`, `status`, `last_accessed`, `time_spent`, `attempts`)

Chei semantice: `userID * lessonID * start_date`

Constrângeri de domeniu:

- ▷ $\text{DOM}(\text{progress}) = \{n \in \text{TINYINT} \mid 0 \leq n \leq 100\} \text{ DEFAULT } 0$
- ▷ $\text{DOM}(\text{status}) = \{n \in \text{NVARCHAR} \mid n \in \{\text{'not_started'}, \text{'in_progress'}, \text{'completed'}\}\} \text{ DEFAULT 'not_started'}$
- ▷ $\text{DOM}(\text{time_spent}) = \text{INT DEFAULT } 0$
- ▷ $\text{DOM}(\text{attempts}) = \text{INT DEFAULT } 0$

Constrângeri de existență: `userID`, `lessonID`

Constrângeri de integritate referențială:

`userID` $\subset$ `USERS.userID`

`lessonID` $\subset$ `LESSONS.lessonID`

Constrângeri de tuplu: `start_date` $<$ `finish_date`

ASSIGNMENTS (`assignmentID`, `title`, `description`, `start_date`, `due_date`, `max_score`, `lessonID`)

Chei semantice: `lessonID * title`

Constrângeri de domeniu:

- ▷ $\text{DOM}(\text{title}) = \text{NVARCHAR}(255)$
- ▷ $\text{DOM}(\text{max_score}) = \{n \in \text{INT} \mid 0 < n \leq 100\}$
- ▷ $\text{DOM}(\text{start_date}) = \text{DATETIME}$
- ▷ $\text{DOM}(\text{due_date}) = \text{DATETIME}$

Constrângeri de existență: `title`, `start_date`, `due_date`, `lessonID`

Constrângeri de integritate referențială: `lessonID` $\subset$ `LESSONS.lessonID`

Constrângeri de tuplu: `start_date` $<$ `due_date`

USER_SUBMISSIONS (`submissionID`, `userID`, `assignmentID`, `submission_date`, `grade`, `file_path`, `comments`, `status`)

Chei semantice: `userID * assignmentID * submission_date`

Constrângeri de domeniu:

- ▷ $\text{DOM}(\text{grade}) = \{n \in \text{DECIMAL}(5, 2) \mid 0 \leq n \leq 100\}$
- ▷ $\text{DOM}(\text{status}) = \{n \in \text{NVARCHAR} \mid n \in \{\text{'pending'}, \text{'graded'}, \text{'late'}\}\} \text{ DEFAULT 'pending'}$

Constrângeri de existență: `userID`, `assignmentID`, `submission_date`

Constrângeri de integritate referențială:

`userID` $\subset$ `USERS.userID`
`assignmentID` $\subset$ `ASSIGNMENTS.assignmentID`

BADGES (badgeID, name, description, icon_path, created_at)

Chei semantice: name

Constrângeri de domeniu:

- ▷ `DOM(name) = NVARCHAR(100)`
- ▷ `DOM(description) = NVARCHAR(500)`
- ▷ `DOM(icon_path) = NVARCHAR(255)`
- ▷ `DOM(created_at) = DATETIME DEFAULT GETDATE()`

Constrângeri de existență: name

USER_BADGES (userID, badgeID, awarded_at)

Chei semantice: userID * badgeID

Constrângeri de domeniu:

- ▷ `DOM(awarded_at) = DATETIME DEFAULT GETDATE()`

Constrângeri de integritate referențială:

`userID` $\subset$ `USERS.userID`
`badgeID` $\subset$ `BADGES.badgeID`

QUIZZES (quizID, lessonID, title, description, duration_minutes, created_at)

Chei semantice: lessonID * title

Constrângeri de domeniu:

- ▷ `DOM(title) = NVARCHAR(200)`
- ▷ `DOM(duration_minutes) = {  $n \in \text{INT} \mid n > 0$  }`
- ▷ `DOM(created_at) = DATETIME DEFAULT GETDATE()`

Constrângeri de existență: lessonID, title

Constrângeri de integritate referențială:

`lessonID` $\subset$ `LESSONS.lessonID`

TAGS (tagID, name)

Chei semantice: name

Constrângeri de domeniu:

- ▷ `DOM(name) = VARCHAR(255)`

Constrângeri de existență: name

LESSON_TAGS (lessonID, tagID)

Chei semantice: lessonID * tagID

Constrângeri de existență: lessonID, tagID

Constrângeri de integritate referențială: `lessonID` $\subset$ `LESSONS.lessonID`
`tagID` $\subset$ `TAGS.tagID`

ITEMS (itemID, content, type, difficulty, created_at)

Chei semantice: content

Constrângeri de domeniu:

- ▷ DOM(content) = NVARCHAR(255)
- ▷ DOM(type) = VARCHAR(50)
- ▷ DOM(created_at) = DATETIME DEFAULT GETDATE()

Constrângeri de existență: content, type

ITEMCHOICES (itemchoicesID, itemID, content, is_correct)

Chei semantice: itemID * content

Constrângeri de domeniu:

- ▷ DOM(content) = NVARCHAR(255)
- ▷ DOM(is_correct) = BIT DEFAULT 0

Constrângeri de existență: itemID, content

Constrângeri de integritate referențială:

itemID $\subset$ ITEMS.itemID

QUIZ_ITEMS (quizID, itemID, position)

Chei semantice: quizID * itemID

Constrângeri de domeniu:

- ▷ DOM(position) = INT

Constrângeri de existență: quizID, itemID, position

Constrângeri de integritate referențială:

quizID $\subset$ QUIZZES.quizID

itemID $\subset$ ITEMS.itemID

QUIZ_RESULTS (resultID, userID, quizID, itemID, is_correct, attempt_date)

Chei semantice: userID * itemID

Constrângeri de domeniu:

- ▷ DOM(is_correct) = BIT
- ▷ DOM(attempt_date) = DATETIME DEFAULT GETDATE()

Constrângeri de existență: userID, quizID, itemID

Constrângeri de integritate referențială:

userID $\subset$ USERS.userID

(quizID, itemID) $\subset$ QUIZ_ITEMS(quizID, itemID)

ITEM_TAGS (itemID, tagID)

Chei semantice: itemID * tagID

Constrângeri de existență: itemID, tagID

Constrângeri de integritate referențială:

itemID $\subset$ ITEMS.itemID

tagID $\subset$ TAGS.tagID

4.6 Implementarea funcționalităților

Subcapitolul acesta descrie procesul de implementare a principalelor funcționalități ale aplicației, de la înregistrarea utilizatorului, la autentificare, resetarea parolei, algoritmul de recomandare a lecțiilor și predicția academică. Scopul implementării este de a dezvolta un sistem capabil să gestioneze date educaționale relevante și să ofere conținut personalizat fiecărui utilizator.

4.6.1 Register

Metoda **RegisterAjax** gestionează înregistrarea unui nou utilizator în aplicație, având atributul **[HttpPost]**. Primește un obiect *User* prin formular **[FromForm]** și trimite un răspuns JSON care reflectă succesul sau eșecul operației.

```
[HttpPost]
0 references
public IActionResult RegisterAjax([FromForm] User user)
{
    try
    {
        _logger.LogInformation("Start RegisterAjax");

        if (!ModelState.IsValid)
        {
            var errors = ModelState.Values.SelectMany(v => v.Errors)
                .Select(e => e.ErrorMessage)
                .ToList();
            _logger.LogWarning("ModelState invalid: {errors}", string.Join(" | ", errors));
            return Json(new { success = false, message = "Datele introduse nu sunt valide: " + string.Join(" | ", errors) });
        }
    }
}
```

Figura 4.9: Validarea modelului

În secțiunea aceasta se verifică dacă modelul *User* primit este valid conform regulilor definite în Model (*User.cs*). Dacă există erori, acestea sunt colectate și raportate în log, dar și informează clientul despre problemele întâmpinate.

```
if (string.IsNullOrEmpty(user.username) || user.username.Length > 25 ||
    !System.Text.RegularExpressions.Regex.IsMatch(user.username, @"^[a-zA-Z0-9_]+$"))
{
    _logger.LogWarning("Username invalid: {username}", user.username);
    return Json(new { success = false, message = "Username invalid. Folosește doar litere, cifre sau '_' și maximum 25 de caractere." });
}

if (string.IsNullOrEmpty(user.email) ||
    !System.Text.RegularExpressions.Regex.IsMatch(user.email, @"^[^@\s]+@[^@\s]+\.[^@\s]+$"))
{
    _logger.LogWarning("Email invalid: {email}", user.email);
    return Json(new { success = false, message = "Emailul introdus nu este valid." });
}

if (string.IsNullOrEmpty(user.password_hash) || user.password_hash.Length > 50 || user.password_hash.Length < 3)
{
    _logger.LogWarning("Parola invalidă: lungime {length}", user.password_hash?.Length);
    return Json(new { success = false, message = "Parola nu este validă." });
}
```

Figura 4.10: Validarea datelor introduse

Aici sunt validate câmpurile esențiale, *username* trebuie să fie nevid și să respecte un format permis, *email* este verificat prin regex pentru a confirma validitate, iar *password* trebuie să aibă o lungime acceptabilă.

```
var existingUser = _context.User.FirstOrDefault(u =>
    u.username == user.username || u.email == user.email);

if (existingUser != null)
{
    _logger.LogWarning("User existent cu același username/email");
    return Json(new { success = false, message = "Username sau email deja utilizat" });
}
```

Figura 4.11: Verificarea existenței unui utilizator

Pentru prevenirea conflictelor și asigurarea unicității datelor, se verifică în baza de date dacă un utilizator cu același username sau email există deja. Dacă este găsit, înregistrarea nouă este respinsă și utilizatorul primește un mesaj informativ.

```
user.password_hash = BCrypt.Net.BCrypt.HashPassword(user.password_hash);
user.created_at = DateTime.Now;

if (string.IsNullOrEmpty(user.role)) user.role = "Student";

_context.User.Add(user);
_context.SaveChanges();

var profile = new Profile
{
    userID = user.userID,
    full_name = "",
    avatar_url = "/images/default-avatar.png"
};

_context.Profile.Add(profile);
_context.SaveChanges();
```

Figura 4.12: Crearea utilizatorului și a profilului asociat

Parola este securizată prin intermediul algoritmului **BCrypt** și este setată data creării contului. Dacă rolul utilizatorului nu este specificat, acesta primește implicit rolul de "Student". Ulterior, datele utilizatorului sunt stocate în baza de date și se creează o entitate Profile asociată utilizatorului, cu valori implicite pentru început.

```
HttpContext.Session.SetString("username", user.username);
HttpContext.Session.SetString("role", user.role);

_logger.LogInformation("Utilizator și profil înregistrate: {username}", user.username);

string? redirectUrl = user.role == "Profesor"
    ? Url.Action("TeacherDashboard", "Dashboard")
    : Url.Action("Dashboard", "Dashboard");

return Json(new
{
    success = true,
    message = "Înregistrare cu succes!",
    redirectUrl = redirectUrl
});
}
```

Figura 4.13: Redirecționarea utilizatorului

Odată ce înregistrarea este finalizată cu succes, informațiile utilizatorului sunt setate în sesiunea curentă, pentru a-l menține autentificat pe tot parcursul navigării în aplicație. În log se înregistrează starea autentificării. În funcție de rolul ales, utilizatorul este trimis către pagina de dashboard corespunzătoare.

```
catch (Exception ex)
{
    _logger.LogError(ex, "Excepție în RegisterAjax");
    return Json(new
    {
        success = false,
        message = "Eroare la înregistrare: " + ex.Message
    });
}
```

Figura 4.14: Tratarea erorilor

Metoda prinde excepțiile, le salvează în log și informează utilizatorul cu un mesaj simplu.

4.6.2 Login

Aplicația utilizează metoda **LoginAjax** pentru autentificarea utilizatorilor într-un mod asincron, fiind marcată cu atributul **[HttpPost]**, care indică faptul că răspunde la cereri de tip **POST**. Metoda primește parametrii *username* și *password* prin formular **[FromForm]**, returnând un răspuns JSON în funcție de rezultatul autentificării.

```
[HttpPost]

0 references
public IActionResult LoginAjax([FromForm] string username, [FromForm] string password)
{
    try
    {
        _logger.LogInformation("Start LoginAjax");

        if (string.IsNullOrEmpty(username) || string.IsNullOrEmpty(password))
        {
            return Json(new
            {
                success = false,
                message = "Username și parola sunt obligatorii."
            });
        }

        if (username.Length > 25 || password.Length > 50)
        {
            return Json(new
            {
                success = false,
                message = "Username sau parola invalida."
            });
        }
    }
}
```

Figura 4.15: Validarea câmpurilor de autentificare

În prima parte a metodei sunt realizate verificări cruciale asupra validității datelor de intrare. În cazul în care oricare dintre câmpurile *username* sau *password* este gol sau depășesc dimensiunile maxime admise, se returnează un mesaj de eroare. Această validare protejează aplicația împotriva trimerii datelor incomplete.

```
var user = _context.User.FirstOrDefault(u => u.username == username);

if (user == null || !BCrypt.Net.BCrypt.Verify(password, user.password_hash))
{
    _logger.LogWarning("Autentificare eșuată pentru: {username}", username);
    return Json(new
    {
        success = false,
        message = "Username sau parolă incorectă."
    });
}
```

Figura 4.16: Verificarea utilizatorului și a parolei

După ce validarea inițială a avut loc, se caută în baza de date un utilizator care să corespundă numelui de utilizator introdus. Dacă numele nu este găsit sau parola introdusă nu corespunde hash-ului salvat (fiind verificat cu ajutorul bibliotecii **BCrypt**), loginul eșuează. Mesajul de eroare este simplu și nu dezvăluie informații suplimentare.

```
user.last_login = DateTime.Now;

user.LoginCount++;

_context.SaveChanges();

HttpContext.Session.SetString("username", user.username);
HttpContext.Session.SetString("role", user.role);

_logger.LogInformation("Autentificare reușită: {username}. Login count: {count}", username, user.LoginCount);
```

Figura 4.17: Actualizarea datelor și inițializarea sesiunii

Atunci când autentificarea este validă, se actualizează în baza de date date ultimei autentificări *last_login* și crește contorul de autentificări *LoginCount*. În urma acestora, username-ul și rolul sunt salvate în sesiunea curentă, fiind necesare pentru persistarea autentificării pe parcursul navigării în aplicație. Rolul utilizatorului este extrem de important deoarece controlează accesul la anumite funcționalități ale aplicației.

```
user.last_login = DateTime.Now;

user.LoginCount++;

_context.SaveChanges();

HttpContext.Session.SetString("username", user.username);
HttpContext.Session.SetString("role", user.role);

_logger.LogInformation("Autentificare reușită: {username}. Login count: {count}", username, user.LoginCount);
```

Figura 4.18: Redirecționarea și tratarea erorilor

În ultima parte a metodei se determină către ce panou de control va fi redirecționat utilizatorul după autentificare, depinzând de rolul său: profesorii sunt trimiși către o pagină dedicată (*TeacherDashboard*), iar studenții sunt redirecționați către panoul de control principal (*Dashboard*). La final, dacă apare vreo excepție în oricare dintre pașii anteriori, este înregistrată în sistemul de loguri și returnează un mesaj de eroare.

4.6.3 Resetarea parolei

Metoda **ResetPasswordAjax** este implementată cu scopul resetării parolei utilizatorilor într-un mod asincron, fiind marcată cu **[HttpPost]**, răspunzând la cereri de tip **POST**. Prin formular, **[FromForm]** se primesc parametrii *email*, *newPassword* și *confirmPassword*. Precum celelalte metode, se returnează un răspuns JSON care indică dacă procesul de resetare a parolei a eșuat sau nu.

În această secțiune sunt efectuate validări asupra datelor introduse de utilizator, se verifică dacă câmpurile sunt completate și dacă adresa de email respectă formatul valid. Aceste verificări previn trimiterea datelor incomplete sau incorecte, iar regex-ul utilizat filtrează emailurile care nu respectă formatul standard.

```
[HttpPost]
0 references
public IActionResult ResetPasswordAjax([FromForm] string email, [FromForm] string newPassword, [FromForm] string confirmPassword)
{
    try
    {
        _logger.LogInformation("Start ResetPasswordAjax");

        if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(newPassword) || string.IsNullOrEmpty(confirmPassword))
        {
            return Json(new
            {
                success = false,
                message = "Toate câmpurile sunt obligatorii."
            });
        }

        if (!System.Text.RegularExpressions.Regex.IsMatch(email, @"^[^@\s]+@([^\s]+\.[^\s]+)$"))
        {
            return Json(new
            {
                success = false,
                message = "Emailul nu este valid."
            });
        }
    }
}
```

Figura 4.19: Validarea câmpurilor pentru resetarea parolei

```
if (newPassword != confirmPassword)
{
    return Json(new
    {
        success = false,
        message = "Parolele nu se potrivesc"
    });
}

if (newPassword.Length > 50)
{
    return Json(new
    {
        success = false,
        message = "Parola este prea lungă. Maximum 50 caractere."
    });
}

if (newPassword.Length < 3)
{
    return Json(new
    {
        success = false,
        message = "Parola este prea scurtă. Minimum 3 caractere."
    });
}
```

Figura 4.20: Verificarea parolelor

În urma validării emailului, urmează verificarea parolelor introduse. Câmpurile *newPassword* și *confirmPassword* sunt comparate, iar dacă acestea nu coincid, un mesaj de eroare este returnat. De asemenea, sunt verificate și constrângerile privind lungimea parolei, trebuie să aibă minim 3 caractere și maxim 50, prevenind parolele slabe și atacurile.

```
var user = _context.User.FirstOrDefault(u => u.email == email);
if (user == null)
{
    return Json(new
    {
        success = false,
        message = "Nu există un utilizator cu acest email."
    });
}
```

Figura 4.21: Căutarea utilizatorului după email

După verificarea parolelor, se identifică în baza de date un utilizator care corespunde adresei de email introduse. Dacă nu există niciun cont asociat emailului furnizat, aplicația returnează un mesaj informativ, fără a afișa alte detalii în plus.

```
user.password_hash = BCrypt.Net.BCrypt.HashPassword(newPassword);
_context.SaveChanges();

_logger.LogInformation("Parolă resetată pentru: {email}", email);
return Json(new
{
    success = true,
    message = "Parola a fost resetată cu succes!",
    redirectUrl = "/Login"
});
}
catch (Exception ex)
{
    _logger.LogError(ex, "Eroare la resetarea parolei");
    return Json(new
    {
        success = false,
        message = "Eroare la resetarea parolei: " + ex.Message
    });
}
```

Figura 4.22: Resetarea parolei și tratarea excepțiilor

Utilizatorul a fost identificat, parola nouă este procesată și stocată în baza de date. Nu este stocată exact așa cum a fost introdusă, ci este securizată prin intermediul funcției *HashPassword* din biblioteca **BCrypt**. Ulterior, evenimentul este înregistrat în sistemul de loguri. În cazul în care parola a fost resetată cu succes, utilizatorul este informat printr-un mesaj de tip JSON și este redirecționat către pagina de login. Experiența utilizatorului este simplă și intuitivă. Cu ajutorul unui bloc *try-catch*, sunt găsite eventualele erori apărute în timpul execuției, fiind urmate de înregistrarea acestora și afișarea unui mesaj de eroare standard.

4.6.4 Recomandarea lecțiilor

Una dintre cele mai importante funcționalități ale aplicației Achieve+ este **recomandarea lecțiilor**, care are rolul de a ghida utilizatorul spre lecții relevante și personalizate pe baza progresului său anterior. Algoritmul folosit este **similitudinea cosinus**, fiind utilizate taguri ale lecțiilor parcurse și ale celor neîncepute.

```
var allTags = await _context.Tags.Select(t => t.tagID).ToListAsync();
var tagIndexMap = allTags.Select((tagId, index) => new { tagId, index })
    .ToDictionary(x => x.tagId, x => x.index);

int vectorLength = allTags.Count;
var userVector = new double[vectorLength];

foreach (var lu in startedLessonsUsers)
{
    if (lu.progress > 0)
    {
        foreach (var lt in lu.Lesson.LessonTags)
        {
            if (lt.Tag != null && tagIndexMap.TryGetValue(lt.Tag.tagID, out int idx))
            {
                userVector[idx] += 1;
            }
        }
    }
}

Normalize(userVector);
```

Figura 4.23: Generarea vectorului pe baza lecțiilor începute

Un vector (*userVector*) este construit pe baza tagurilor lecțiilor parcurse, denotând interesele studentului. Fiecare poziție din vector corespunde unui tag, iar valoarea este frecvența apariției aceluși tag în lecțiile parcurse. În final, vectorul este normalizat pentru a putea fi comparat prin similitudine cosinus.

```
var unstartedLessons = await _context.Lesson
    .Include(l => l.LessonTags)
    .ThenInclude(lt => lt.Tag)
    .Where(l => !startedLessonIDs.Contains(l.lessonID))
    .ToListAsync();

var recommendations = new List<(Lesson lesson, double score)>();
```

Figura 4.24: Generarea vectorului pe baza lecțiilor începute

Toate lecțiile încă neîncepute de către utilizator sunt selectate, fiind potențialele lecții recomandate care pot contribui la progresul acestuia. O listă de recomandări este inițializată, unde fiecare element este o pereche formată dintr-o lecție și scorul asociat.

```
foreach (var lesson in unstartedLessons)
{
    var lessonVector = new double[vectorLength];

    foreach (var lt in lesson.LessonTags)
    {
        if (lt.Tag != null && tagIndexMap.TryGetValue(lt.Tag.tagID, out int idx))
        {
            lessonVector[idx] = 1;
        }
    }

    Normalize(lessonVector);
    var similarity = CosineSimilarity(userVector, lessonVector);

    if (similarity > 0)
    {
        recommendations.Add((lesson, similarity));
    }
}
```

Figura 4.25: Generarea vectorului pe baza lecțiilor neîncepute

Pentru fiecare lecție neîncepută, se construiește un vector binar: pozițiile corespunzătoare tagurilor asociate unei lecții sunt setate cu 1. După normalizare, este calculată similitudinea cosinus între vectorul lecției și vectorul utilizatorului. Dacă scorul este pozitiv, lecția este considerată relevantă și adăugată în lista de recomandări.

```
List<LessonProgressViewModel> topRecommendations;

if (recommendations.Count > 0)
{
    topRecommendations = recommendations
        .OrderByDescending(r => r.score)
        .Take(5)
        .Select(r => new LessonProgressViewModel
        {
            LessonID = r.lesson.lessonID,
            LessonTitle = r.lesson.lesson_title,
            LessonContent = r.lesson.lesson_content,
            Progress = 0,
            Status = "not_started",
            Tags = r.lesson.LessonTags
                .Where(lt => lt.Tag != null)
                .Select(lt => lt.Tag.name)
                .ToList(),
            ModuleID = r.lesson.moduleID
        })
        .ToList();
}
```

Figura 4.26: Selectarea celor mai relevante lecții

Lecțiile recomandate sunt sortate în mod descrescător în funcție de scor, iar primele 5 sunt transformate în obiecte de tip *LessonProgressViewModel* pentru afișare.

```
else
{
    topRecommendations = unstartedLessons
        .Take(5)
        .Select(l => new LessonProgressViewModel
        {
            LessonID = l.lessonID,
            LessonTitle = l.lesson_title,
            LessonContent = l.lesson_content,
            Progress = 0,
            Status = "not_started",
            Tags = l.LessonTags
                .Where(lt => lt.Tag != null)
                .Select(lt => lt.Tag.name)
                .ToList(),
            ModuleID = l.moduleID
        })
        .ToList();

    ViewBag.RecommendedLessons = topRecommendations;
}
```

Figura 4.27: Lecțiile neîncepute

În scenariul în care utilizatorul abia s-a logat sau nu a parcurs nicio lecție, sunt returnate primele 5 lecții neîncepute, fără a se baza pe vreun algoritm de recomandare. Rezultatul final, cele mai bune lecții pentru utilizator, este transmis prin *ViewBag* pentru afișarea în interfață.

```
2 reference
private void Normalize(double[] vector)
{
    var norm = Math.Sqrt(vector.Sum(x => x * x));
    if (norm > 0)
    {
        for (int i = 0; i < vector.Length; i++)
        {
            vector[i] /= norm;
        }
    }
}
```

Figura 4.28: Funcția de normalizare

Această funcție normalizează un vector numeric, împărțind fiecare componentă la norma vectorului. Astfel, vectorul final are lungimea 1, lucru esențial pentru compararea acestora.

```
1 reference
private double CosineSimilarity(double[] vec1, double[] vec2)
{
    double dotProduct = 0;
    double normA = 0;
    double normB = 0;

    for (int i = 0; i < vec1.Length; i++)
    {
        dotProduct += vec1[i] * vec2[i];
        normA += vec1[i] * vec1[i];
        normB += vec2[i] * vec2[i];
    }

    if (normA == 0 || normB == 0)
        return 0;

    return dotProduct / (Math.Sqrt(normA) * Math.Sqrt(normB));
}
```

Figura 4.29: Funcția similitudinii cosinus

Funcția **CosineSimilarity** măsoară cât de similar este un vector al lecției în comparație cu vectorul utilizatorului. Sunt calculate produsul scalar și normele vectorilor, rezultatul fiind cuprins între 0 (nicio similitudine) și 1 (similitudine maximă).

4.6.5 Predicția performanței

Funcționalitatea principală a platformei educaționale Achieve+ este predicția performanței academice a studenților, fiind realizată prin mai mulți pași: extragerea caracteristicilor relevante din baza de date, transmiterea acestora către un serviciu extern de Machine Learning implementat în Flask/Python și, ulterior, preluarea și integrarea răspunsului oferit de model în interfața aplicației. Comunicarea între componente se face prin API REST cu mesaje în format JSON.

```
var predictionRequest = new { features = features };

HttpResponseMessage response;
try
{
    _logger.LogInformation("Sending prediction request to Flask API");
    response = await _httpClient.PostAsJsonAsync("http://localhost:8000/predict", predictionRequest);

    if (response == null)
    {
        _logger.LogError("No response from prediction service");
        throw new Exception("Prediction service unavailable");
    }

    if (!response.IsSuccessStatusCode)
    {
        var errorContent = await response.Content.ReadAsStringAsync();
        _logger.LogError($"Prediction service error: {response.StatusCode} - {errorContent}");
        throw new Exception($"Prediction service error: {response.StatusCode}");
    }
}
catch (HttpRequestException ex)
{
    _logger.LogError($"HTTP Request failed: {ex.Message}");
    return StatusCode(503, "Prediction service unavailable");
}
```

Figura 4.30: Comunicarea cu serviciul extern

În această secvență de cod se trimit caracteristicile (*features*) către un API Flask pentru predicții și se gestionează răspunsul. În cazul în care cererea eșuează sau API-ul nu răspunde, eroarea este înregistrată și se afișează un mesaj adecvat, iar în caz contrar răspunsul JSON este convertit în obiect de tip *PredictionResponse*.

```
4 references
private async Task<List<object>> BuildFeaturesFromDb(int userId)
{
    _logger.LogInformation($"Building features for user {userId}");
    var features = new List<object>();
    const double threshold = 100.0;

    try
    {
        int loginCount = await _context.User
            .Where(u => u.userID == userId)
            .Select(u => u.LoginCount)
            .FirstOrDefaultAsync();
        features.Add((double)loginCount);

        double totalAttempts = await _context.LessonsUsers
            .Where(lu => lu.userID == userId)
            .SumAsync(lu => lu.attempts);
        features.Add(totalAttempts);

        var avgQuizAttempts = await _context.QuizResults
            .Where(qr => qr.userID == userId)
            .GroupBy(qr => qr.quizID)
            .Select(g => (double)g.Count())
            .DefaultIfEmpty()
            .AverageAsync();
        features.Add(avgQuizAttempts);
    }
}
```

Figura 4.31: Metoda "BuildFeaturesFromDb"

Metoda prezentată mai sus extrage și construiește caracteristicile (*features*) necesare, folosind informații din baza de date. Se determină numărul de autentificări, numărul de

încercări ale lecțiilor și media încercărilor testelor. Valorile colectate sunt adunate într-o listă de caracteristici, pregătită pentru transmiterea către serviciul extern.

```
var durations = features.Skip(6).Take(3).Cast<double>().ToList();
double avgDuration = durations.Any() ? durations.Average() : 0;
features.Add(avgDuration);

double totalTimeSpent = await _context.LessonsUsers
    .Where(lu => lu.userID == userId)
    .SumAsync(lu => lu.time_spent);
double engagementScore = 0.4 * totalAttempts + 0.4 * totalTimeSpent + 0.2 * loginCount;
string engagementLevelHL = engagementScore >= threshold ? "H" : "L";
features.Add(engagementLevelHL);
```

Figura 4.32: Metoda "BuildFeaturesFromDb" - continuare

În această secțiune a metodei sunt calculate perioada necesară pentru rezolvarea temelor, sunt extrase ultimele trei valori și se face media dintre ele. Se determină timpul total dedicat parcurgerii lecțiilor prin însumarea tuturor sesiunilor înregistrate în baza de date. În urma acestor caracteristici, se calculează *engagementScore* care este compus din *totalAttempts*, *totalTimeSpent* și *loginCount*. Pragul stabilit pentru implicare de 100 și pe baza acestuia implicarea este clasificată drept "High"(H) sau "Low"(L), esențial pentru stabilirea abandonului.

```
lin_reg_grade = LinearRegression()
lin_reg_grade.fit(X_scaled, y_grade)

log_reg = LogisticRegression(max_iter=1000, random_state=42, class_weight=class_weight_dict)
log_reg.fit(X_scaled, y_dropout)
```

Figura 4.33: Antrenarea modelelor pentru predicție

Acest fragment face parte din codul implementat în Python și antrenează două modele de învățare automată, folosind biblioteca *scikit-learn*. Primul model este o regresie liniară standard, utilizat pentru estimarea notei finale a studentului, iar al doilea model este o regresie logistică destinat predicției riscului de abandon școlar. Modelele sunt antrenate pe seturile de date prezentate în capitolul următor.

```
public class PredictionResponse
{
    4 references
    public double predicted_grade { get; set; }
    5 references
    public double predicted_dropout { get; set; }
    3 references
    public double dropout_probability { get; set; }
    0 references
    public Dictionary<string, double> module_predictions { get; set; }
}
```

Figura 4.34: Modelul PredictionResponse

Menționată mai sus, acest model este conceput pentru a structura răspunsurile modelelor de învățare, fiind utilizată în cadrul API-ului pentru a transmite datele către aplicația client.

Capitolul 5

Prezentarea aplicației

5.1 Utilizarea aplicației

Pagina principală a aplicației prezintă o interfață simplă și intuitivă, utilizatorii având acces la liste cu modulele și lecțiile disponibile, fără autentificare. În schimb, parcurgerea unei lecții sau a unui modul este condiționată de login. 5.1, 5.2, 5.3, 5.4

În urma apăsării pe butonul de register, utilizatorul este redirecționat către un formular, unde trebuie să completeze un username, parolă, email și rolul acestuia (student sau profesor). Dacă utilizatorul apasă pe butonul Login, va fi trimis la o pagină unde își va introduce credențialele (username și parolă). În cazul uitării parolei, acest lucru se poate face prin apăsarea butonului aferent. 5.5, 5.6

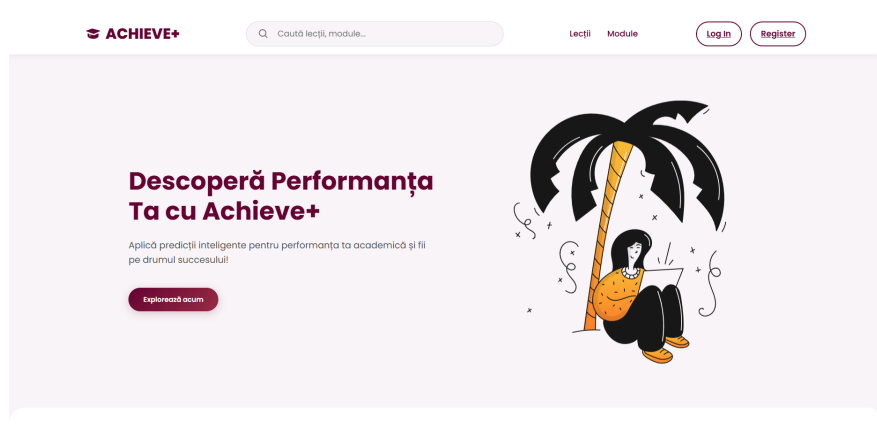


Figura 5.1: Prima parte a paginii principale

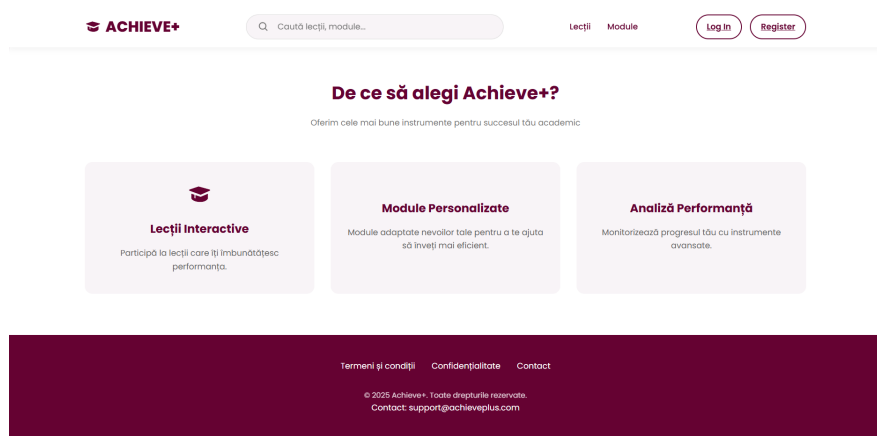


Figura 5.2: A doua parte a paginii principale

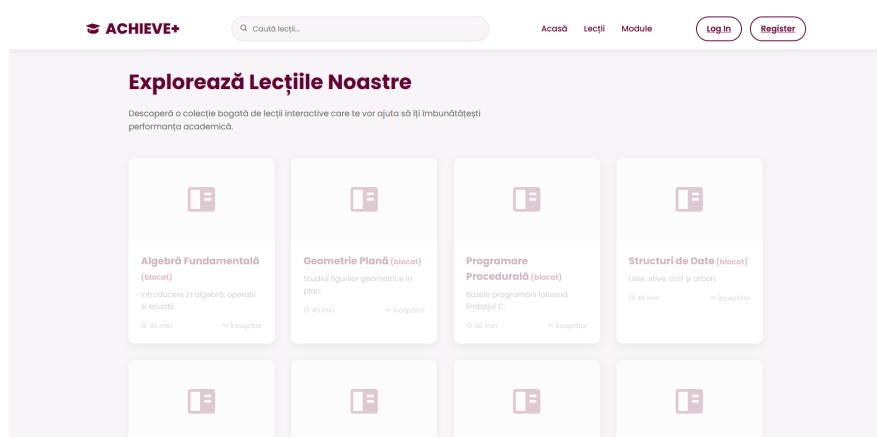


Figura 5.3: Pagina lecțiilor, fără autentificare

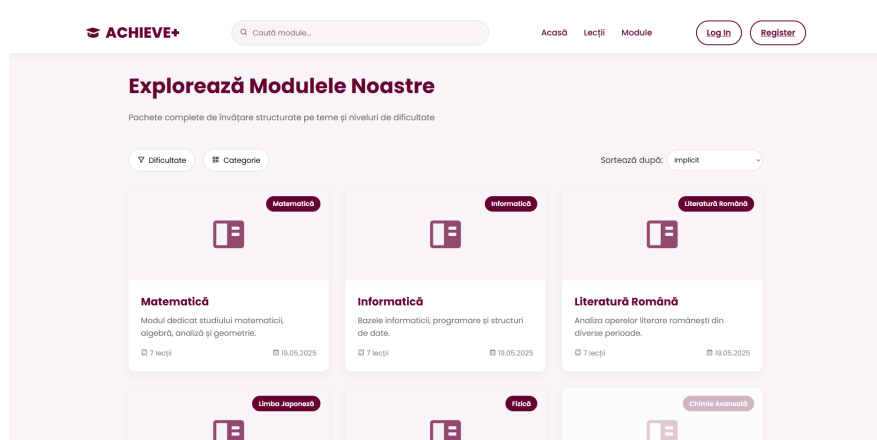
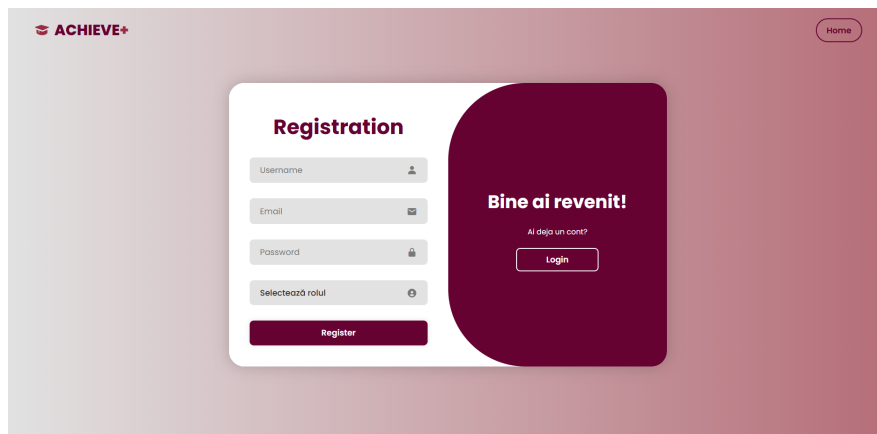
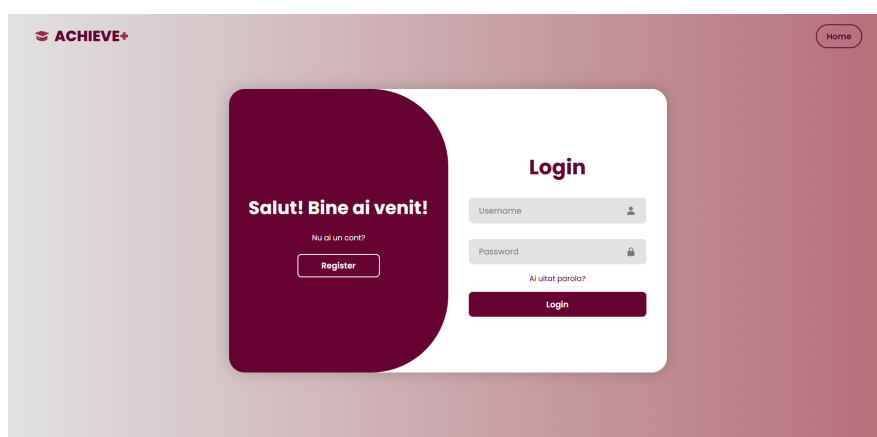


Figura 5.4: Pagina modulelor, fără autentificare



The screenshot shows the 'Registration' page of the ACHIEVE+ application. The page has a dark red background. In the top left corner is the ACHIEVE+ logo, and in the top right corner is a 'Home' button. The main content area is a white card with a dark red header 'Registration'. Below the header are four input fields: 'Username' (with a person icon), 'Email' (with an envelope icon), 'Password' (with a lock icon), and 'Selectează rolul' (with a role icon). Below these fields is a dark red 'Register' button. To the right of the input fields is a dark red rounded rectangle with the text 'Bine ai revenit!' and 'Ai deja un cont?' followed by a 'Login' button.

Figura 5.5: Pagina de Register



The screenshot shows the 'Login' page of the ACHIEVE+ application. The page has a dark red background. In the top left corner is the ACHIEVE+ logo, and in the top right corner is a 'Home' button. The main content area is a white card with a dark red header 'Login'. Below the header are two input fields: 'Username' (with a person icon) and 'Password' (with a lock icon). Below these fields is a dark red 'Login' button. To the left of the input fields is a dark red rounded rectangle with the text 'Salut! Bine ai venit!' and 'Nu ai un cont?' followed by a 'Register' button.

Figura 5.6: Pagina de Login

5.1.1 Student

Dacă utilizatorul a reușit să se înregistreze/logheze cu succes, i se va afișa pagina Dashboard, având trecut câte lecții a parcurs, progresul și testele finalizate. În partea din stânga avem toate butoanele necesare navigării fluente și intuitive: module, lecții, teste, teme, badges, progres și deconectare. În dreapta sus se află username-ul și poza acestuia. Conținutul paginii este simplu, sunt prezentate lecțiile în desfășurare și cele recomandate, personalizate pentru fiecare profil. La apăsarea butonului de deconectare, utilizatorul va fi redirecționat către pagina de login.5.7

Următoarea secțiune este cea a modulelor, întâi se vor afișa toate modulele, apoi la apăsarea butonului se pot vedea detaliile acestora. Fiecare modul conține o descriere, lecțiile și un buton care redirecționează utilizatorul către ele.5.8,5.9

Secțiunea lecțiilor este asemănătoare cu cea a modulelor, întâi avem o listă cu toate lecțiile disponibile, apoi se poate începe parcurgerea lecției. Dacă lecția este finalizată, există un buton de reîncepere a lecției.5.10, 5.11

Testele sunt de trei tipuri - normale, parțial(midterm) și examen final. Fiecare modul are un singur parțial, un singur examen și mai multe teste normale. După finalizarea unui test, studentul poate vedea nota obținută și nu mai poate accesa din nou același test. 5.12,5.13, 5.14

Temele sunt organizate pe aceeași structură precum celelalte secțiuni, se pot vedea toate temele pentru modulele începute, studentul poate încarca fișiere și va primi o notă și feedback.5.15, 5.16

Insignele reprezintă un mod de a motiva studenții, această pagină fiind dedicată vizualizării insinelor obținute și a celor nedebluate încă.5.17

Pagina progresului prezintă statistici, un grafic cu progresul efectuat și predicția performanței, alături de predicția abandonului școlar.5.18,5.19,5.21

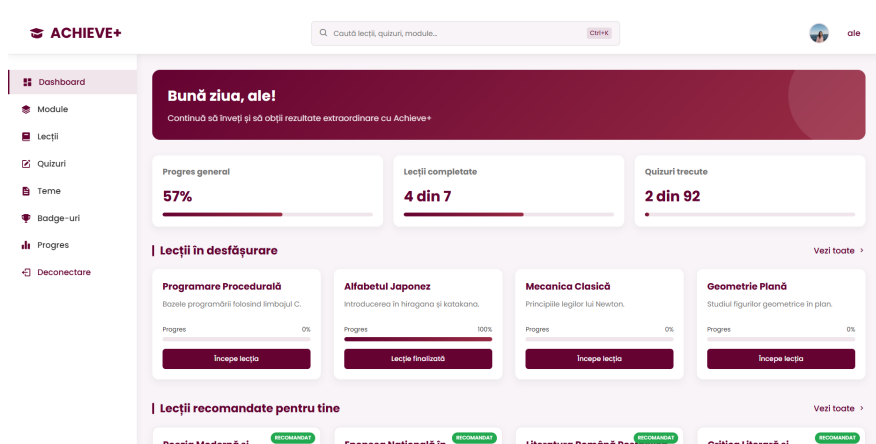


Figura 5.7: Pagina Dashboard

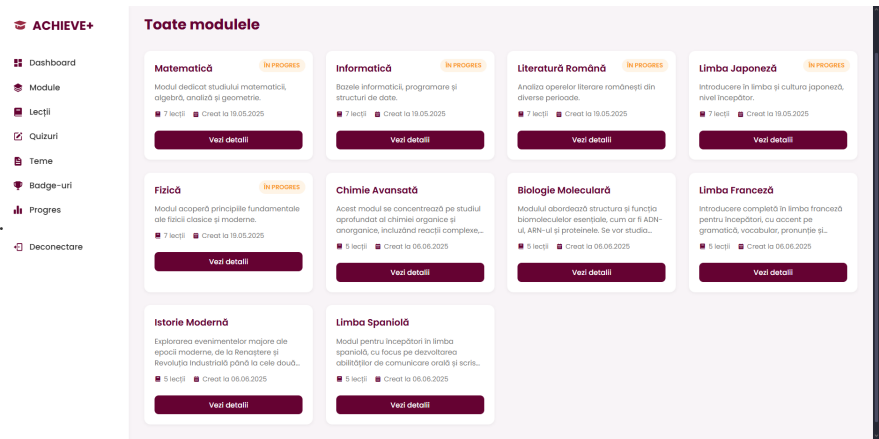


Figura 5.8: Pagina modulelor

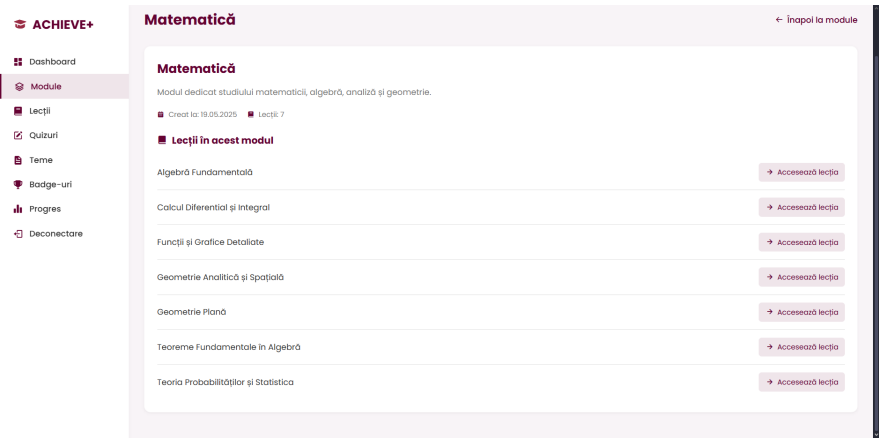


Figura 5.9: Detaliile unui modul

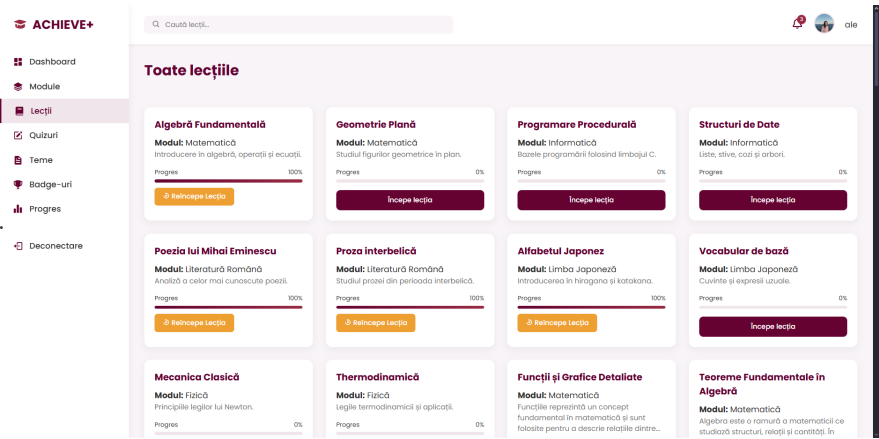


Figura 5.10: Pagina lecțiilor

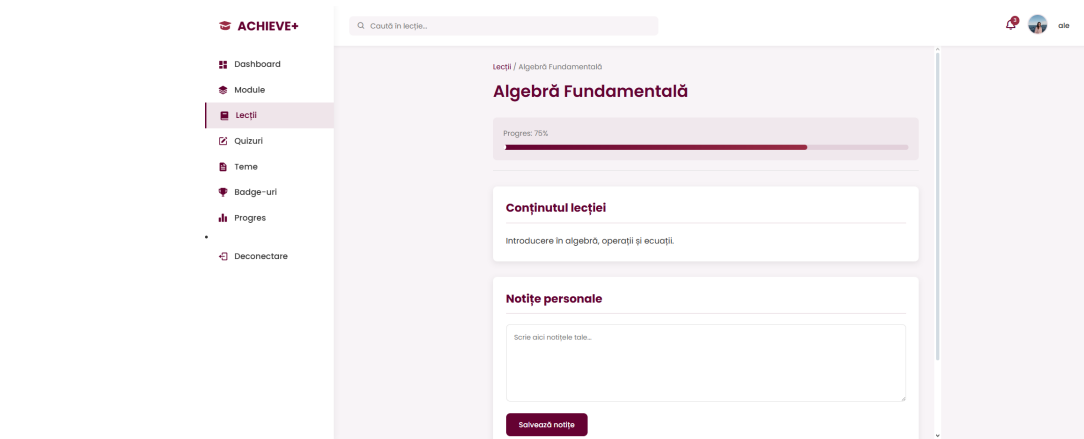


Figura 5.11: Detaliile unei lecții

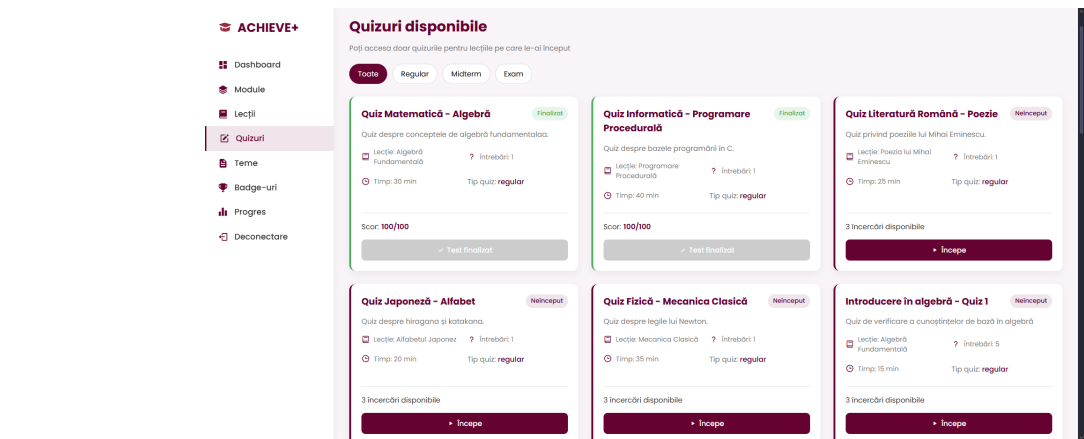


Figura 5.12: Pagina testelor

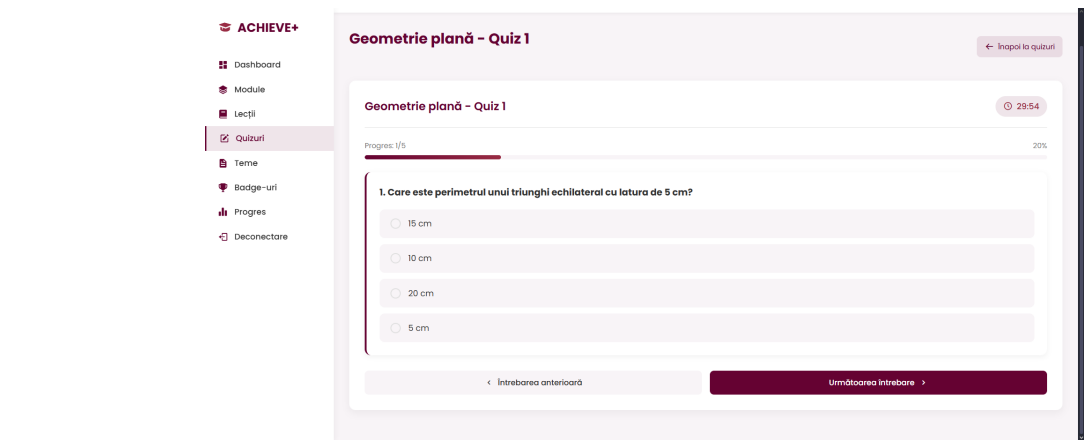


Figura 5.13: Conținutul unui test

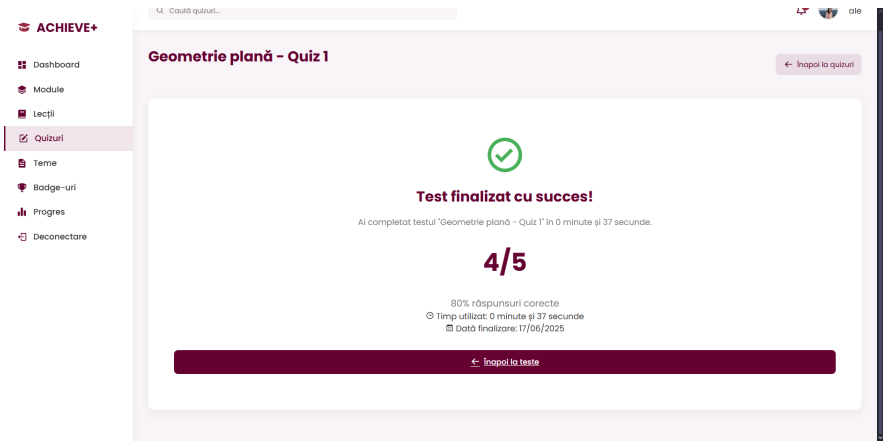


Figura 5.14: Finalizarea unui test

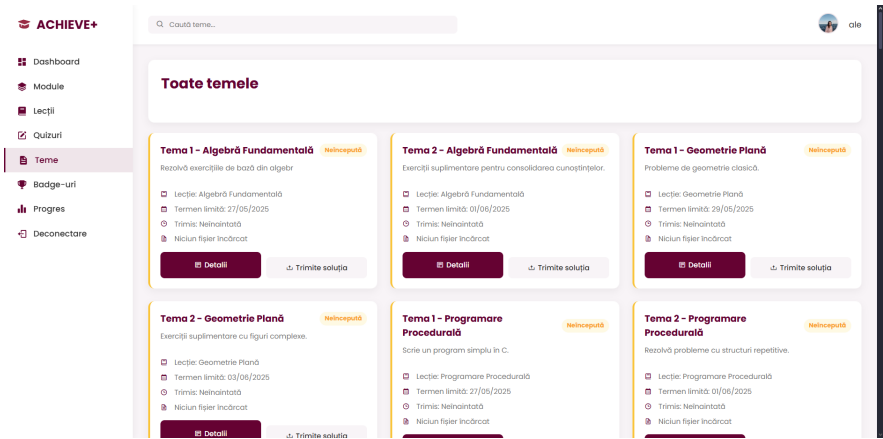


Figura 5.15: Pagina temelor

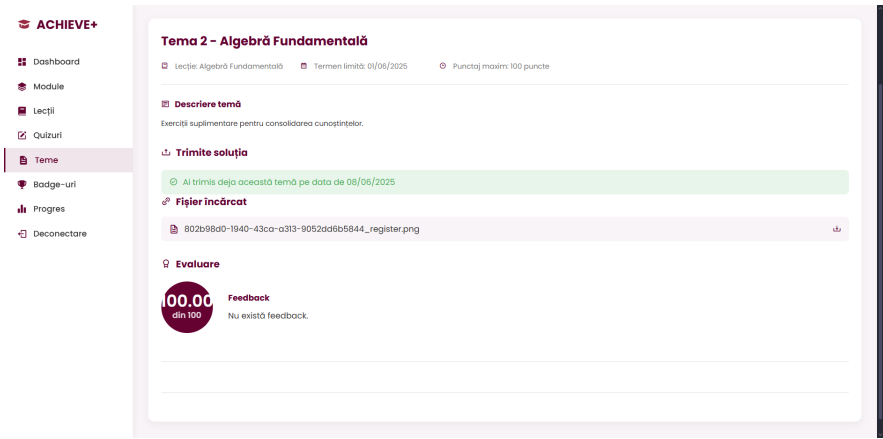


Figura 5.16: Detaliile unei teme

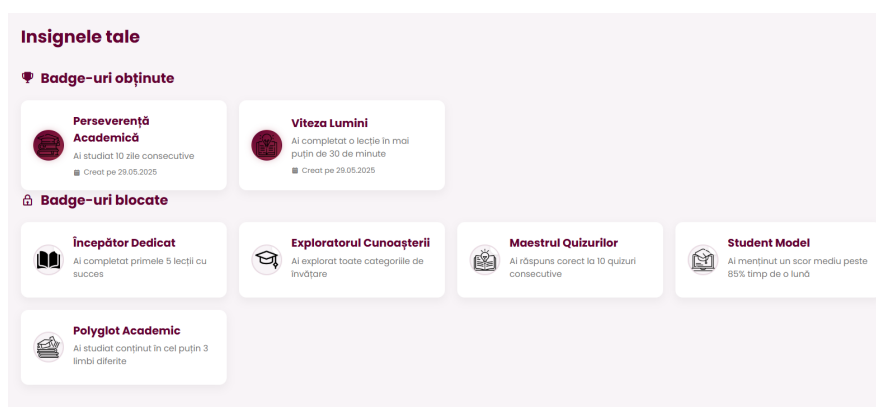


Figura 5.17: Pagina insinelor

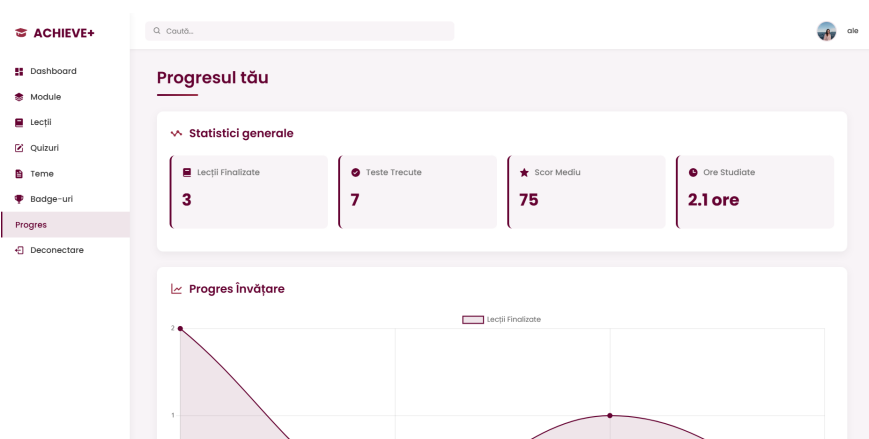


Figura 5.18: Pagina progresului

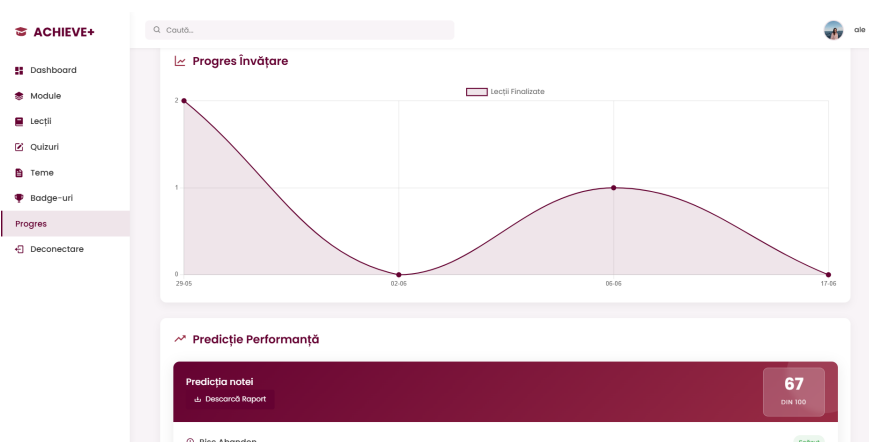


Figura 5.19: Pagina progresului

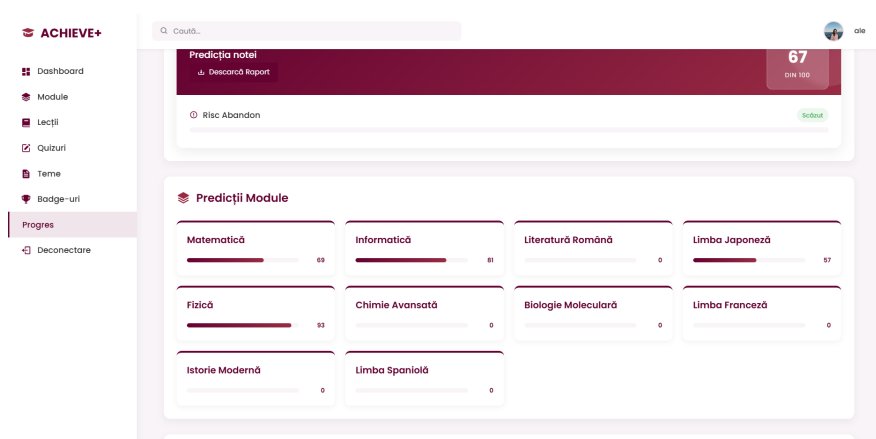


Figura 5.20: Pagina progresului

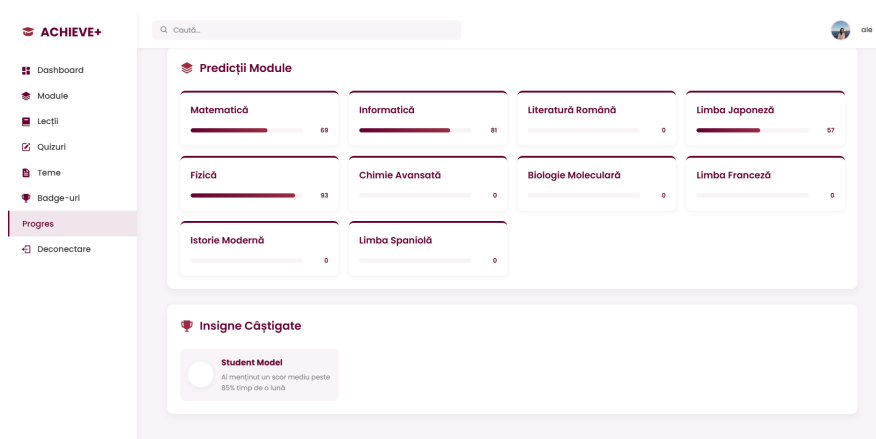


Figura 5.21: Pagina progresului

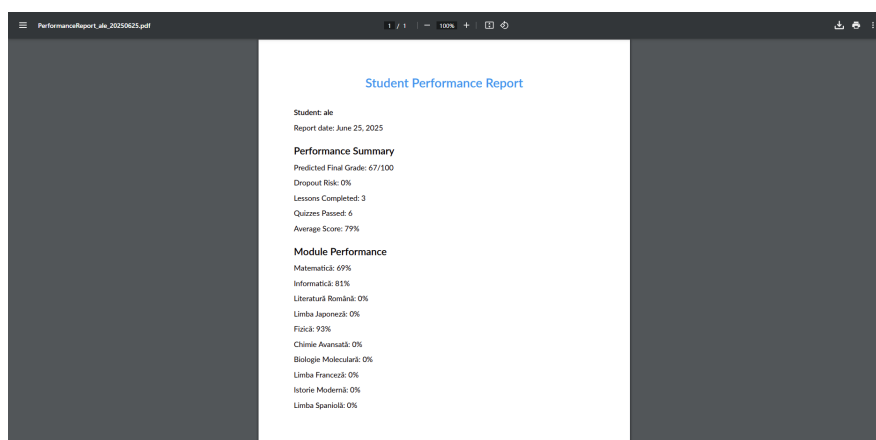


Figura 5.22: Raportul unui student

5.1.2 Profesor

Partea administrativă a acestei aplicații este gestionată de către profesor, care poate crea, modifica și șterge module, lecții, teste, teme. Acesta poate acorda note, feedback și poate vizualiza progresul studenților și notele prezise. În ceea ce privește testele, profesorii pot alege dintre întrebările deja existente sau propune unele noi, incluzând și variante de răspuns. În pagina de progres, aceștia pot vizualiza statistici atât statistici despre fiecare student, cât și notele prezise/abandonul școlar prezis și pot descărca un raport al clasei sau al unui student individual.

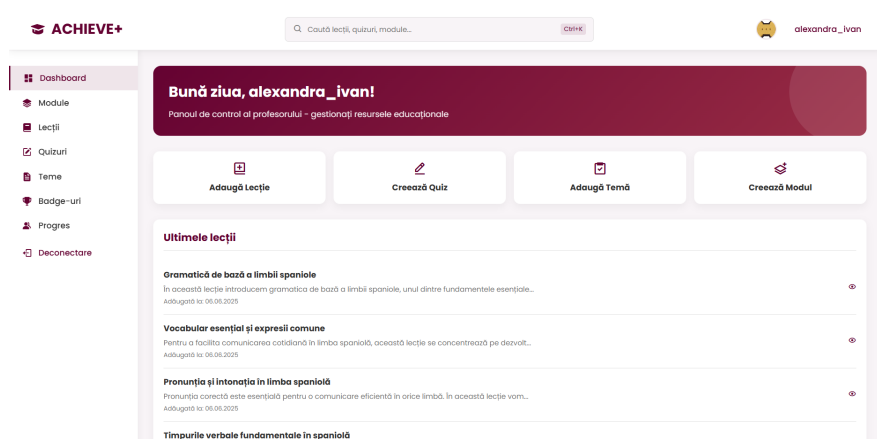


Figura 5.23: Pagina Dashboard pentru profesori

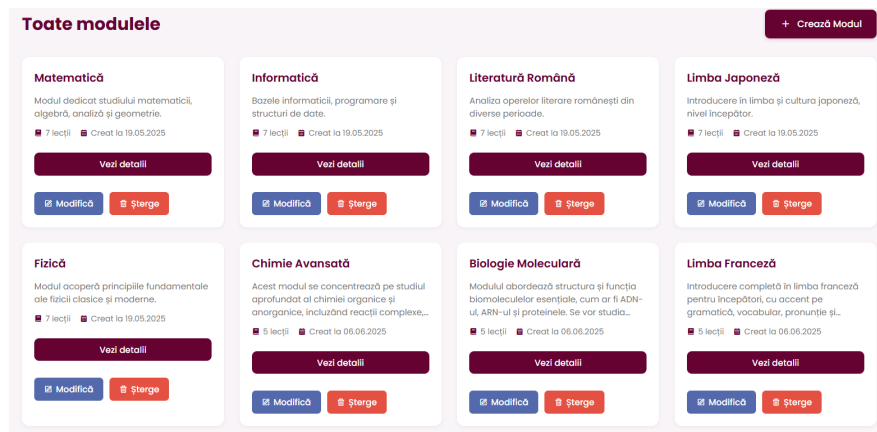
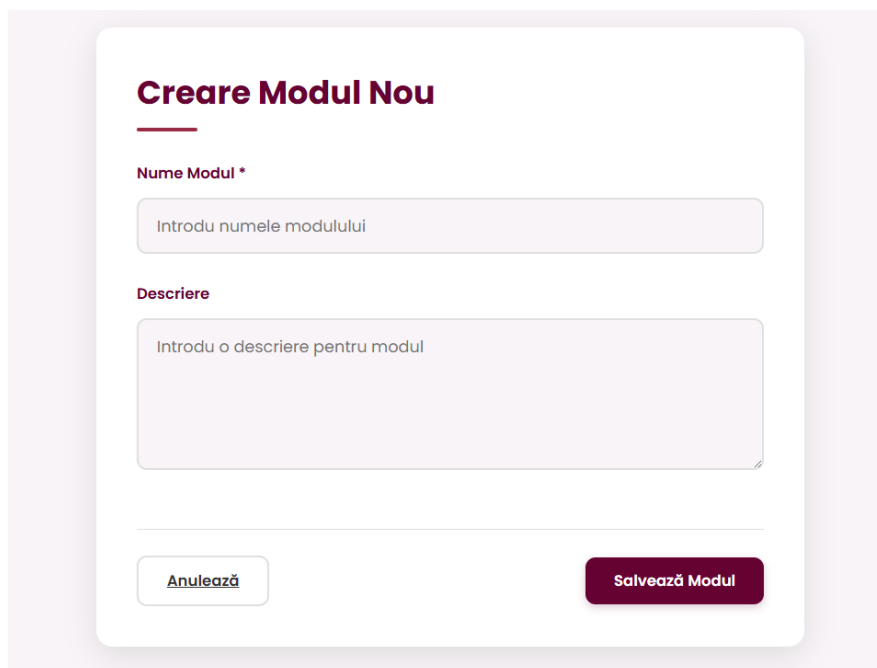
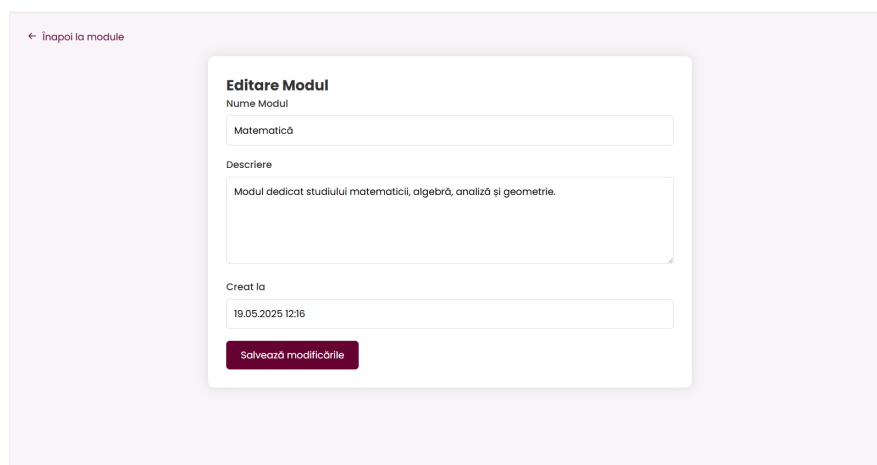


Figura 5.24: Pagina modulelor din perspectiva profesorului



The screenshot shows a form titled "Creare Modul Nou" (Create New Module). It has two main sections: "Nume Modul *" (Module Name) and "Descriere" (Description). The "Nume Modul *" section has a text input field with the placeholder text "Introdu numele modulului". The "Descriere" section has a larger text area with the placeholder text "Introdu o descriere pentru modul". At the bottom of the form, there are two buttons: "Anulează" (Cancel) and "Salvează Modul" (Save Module).

Figura 5.25: Crearea unui modul



The screenshot shows a form titled "Editare Modul" (Edit Module). It has three main sections: "Nume Modul" (Module Name), "Descriere" (Description), and "Creat la" (Created at). The "Nume Modul" section has a text input field with the value "Matematică". The "Descriere" section has a larger text area with the value "Modul dedicat studiului matematicii, algebră, analiză și geometrie.". The "Creat la" section has a text input field with the value "19.05.2025 12:16". At the bottom of the form, there is a button labeled "Salvează modificările" (Save changes).

Figura 5.26: Modificarea unui modul



Figura 5.27: Ștergerea unui modul

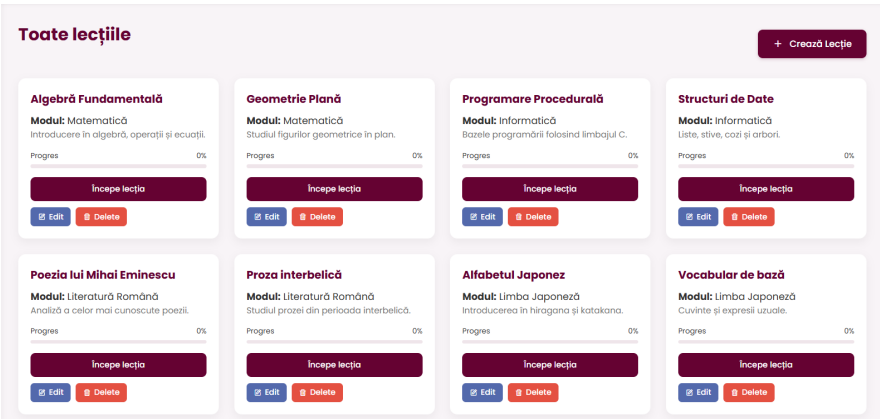


Figura 5.28: Pagina lecțiilor din perspectiva profesorului

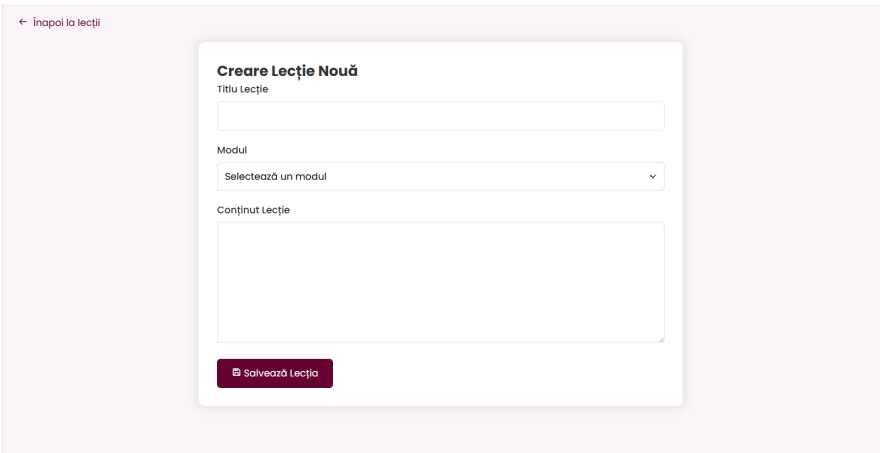


Figura 5.29: Crearea unei lecții

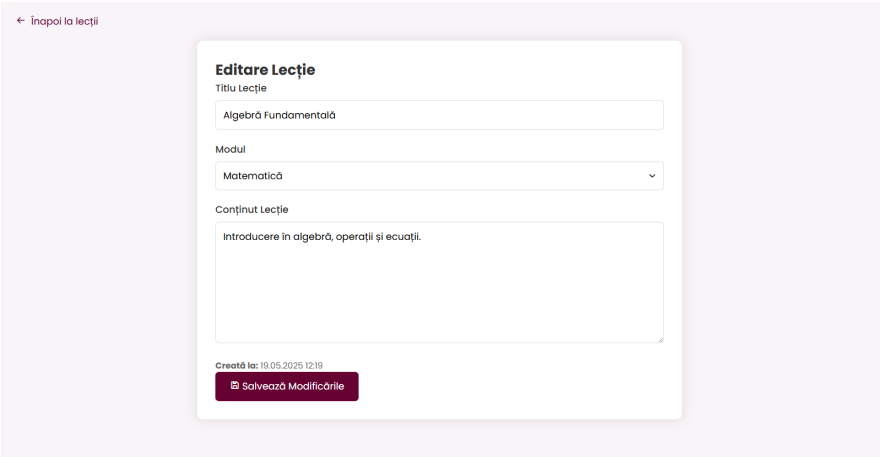


Figura 5.30: Modificarea unei lecții

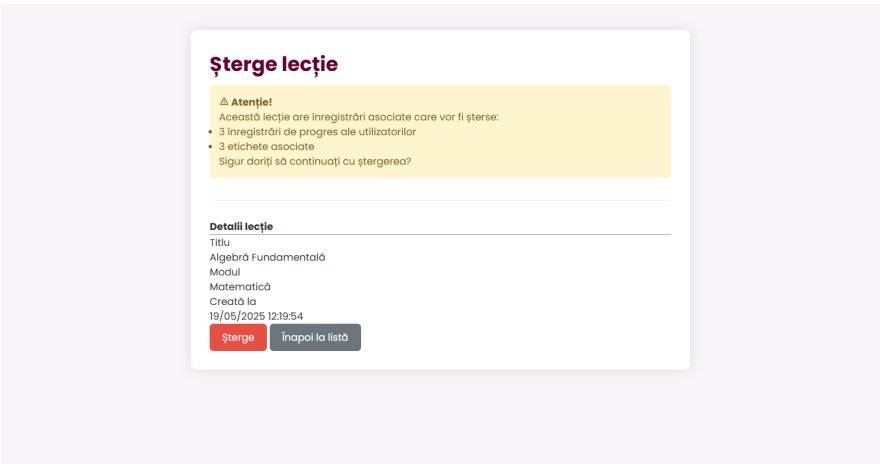


Figura 5.31: Ștergerea unei lecții

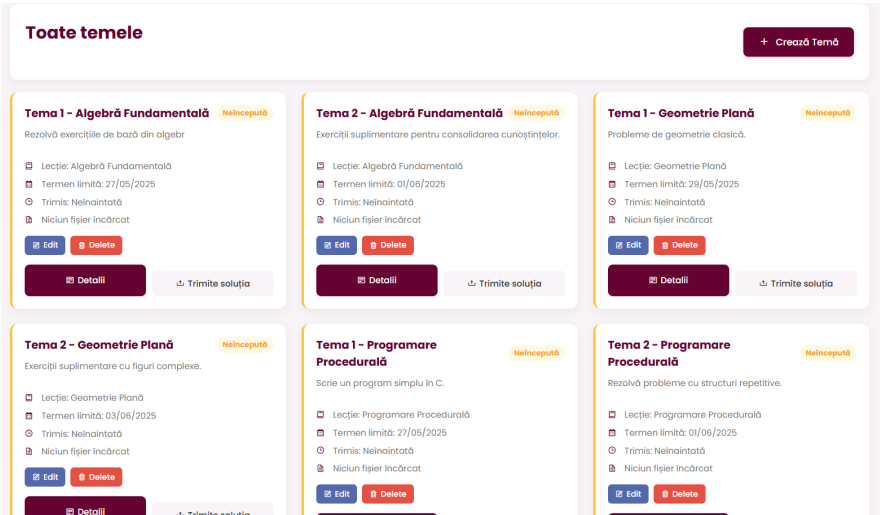
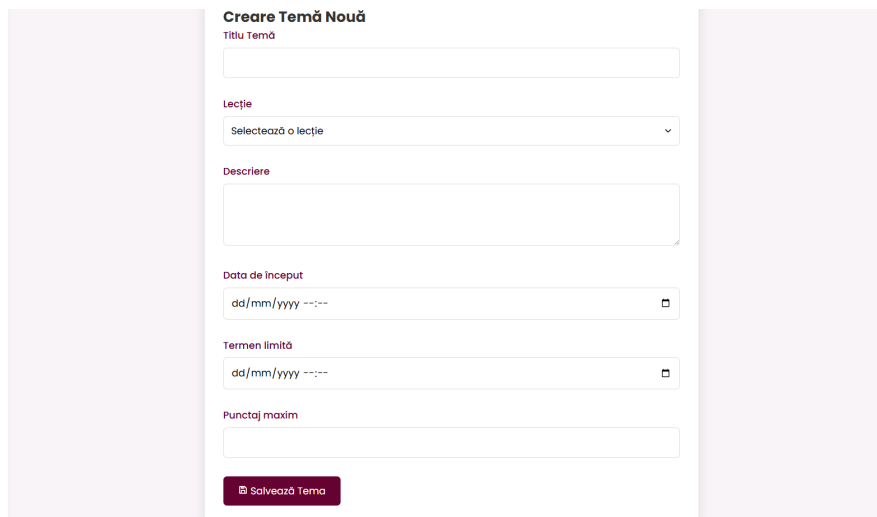


Figura 5.32: Pagina temelor din perspectiva profesorului



Creare Temă Nouă

Titlu Temă

Lecție

Descriere

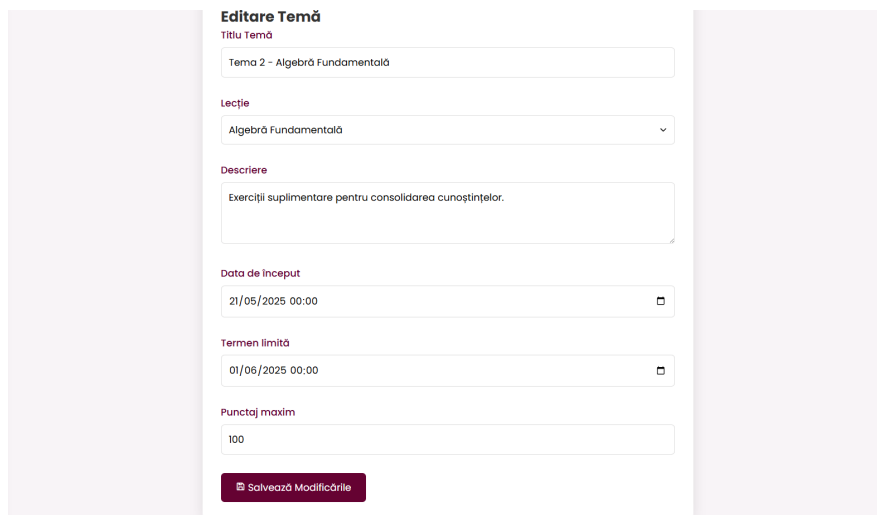
Data de început

Termen limită

Punctaj maxim

Salvează Tema

Figura 5.33: Crearea unei teme



Editare Temă

Titlu Temă

Lecție

Descriere

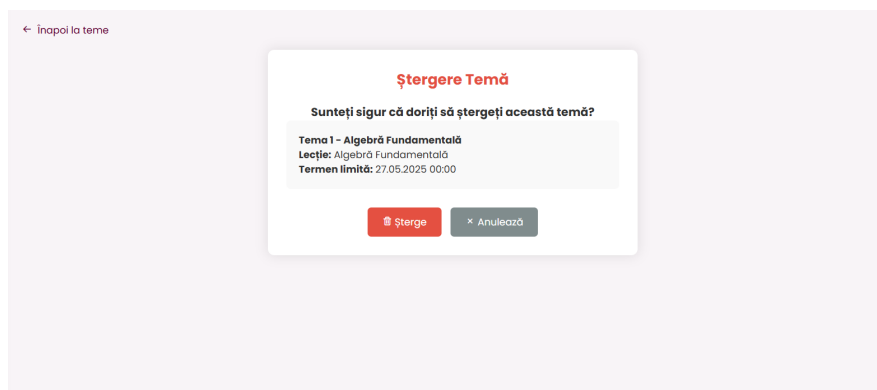
Data de început

Termen limită

Punctaj maxim

Salvează Modificările

Figura 5.34: Modificarea unei teme



← Înapoi la teme

Ștergere Temă

Sunteți sigur că doriți să ștergeți această temă?

Tema 1 - Algebră Fundamentală
Lecție: Algebră Fundamentală
Termen limită: 27.05.2025 00:00

Șterge Anulează

Figura 5.35: Ștergerea unei teme

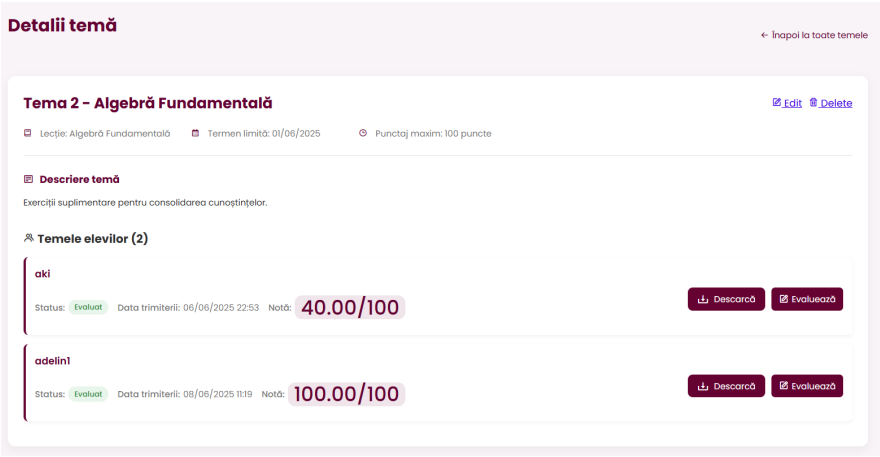


Figura 5.36: Vizualizarea detaliilor unei teme

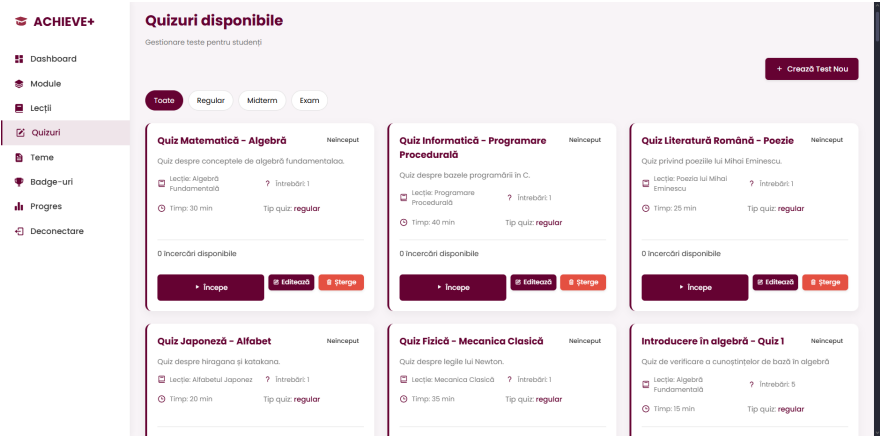


Figura 5.37: Vizualizarea testelor din perspectiva profesorului

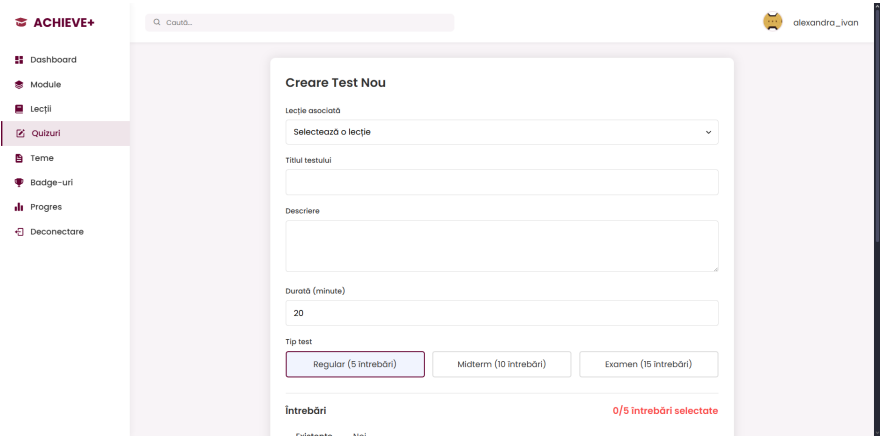


Figura 5.38: Crearea unui test

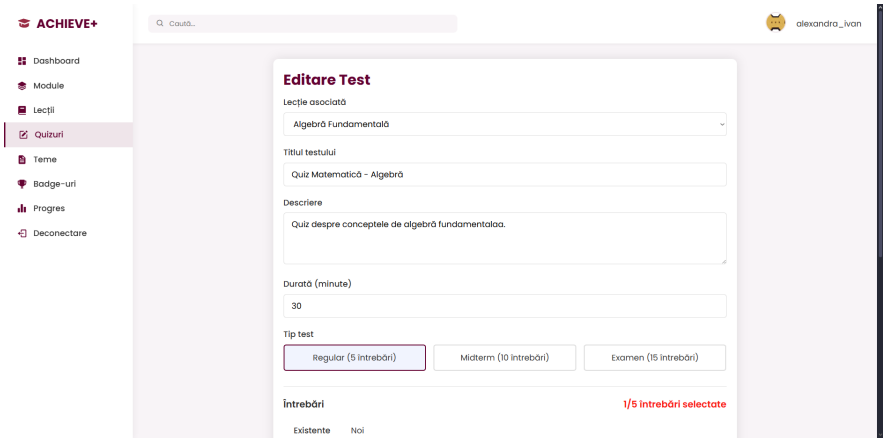


Figura 5.39: Modificarea unui test



Figura 5.40: Ștergerea unui test

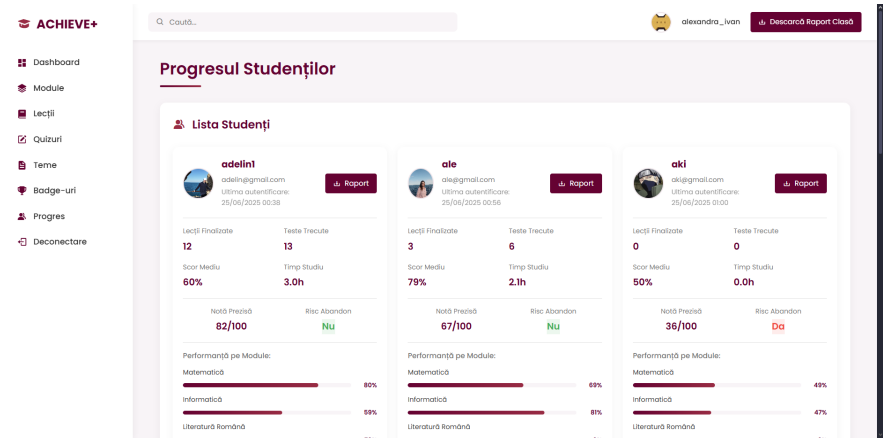


Figura 5.41: Pagina de progres din perspectiva profesorului

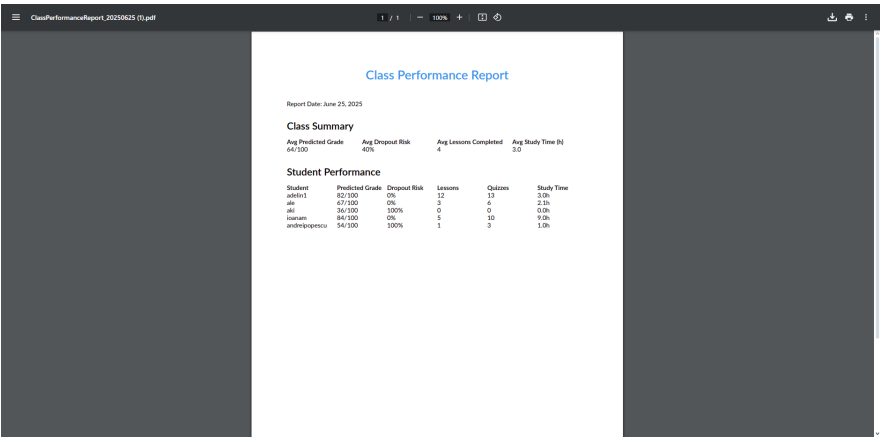


Figura 5.42: Raportul clasei

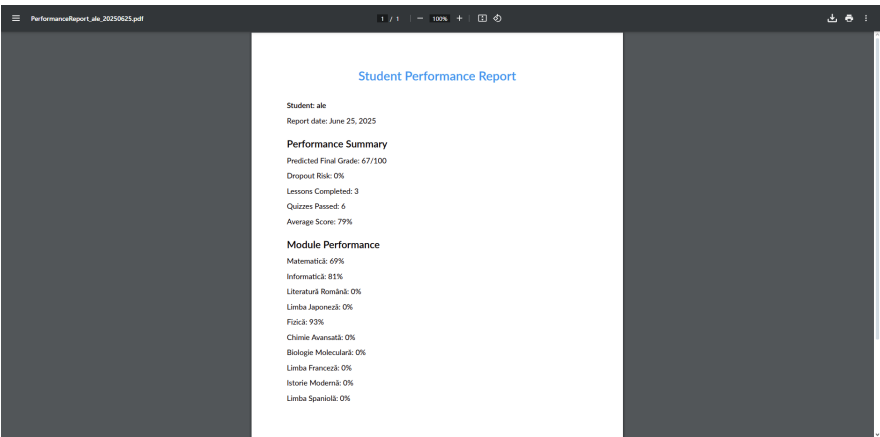


Figura 5.43: Raportul unui student

Capitolul 6

Concluzii

Dezvoltarea acestei aplicații a avut ca scop principal îmbunătățirea procesului de învățare prin adaptarea conținutului pentru nevoile fiecărui utilizator. Consider că, prin recomandarea personalizată a lecțiilor, evaluarea progresului și estimarea notelor, am reușit să ofer utilizatorului un mediu prietenos și motivant, încurajând implicarea în procesul educațional. Obiectivele propuse au fost realizate cu succes, iar platforma răspunde atât cerințelor funcționale, cât și celor nefuncționale, furnizând o experiență intuitivă.

Pe viitor, intenționez să extind funcționalitățile platformei cu elemente care vor sprijini comunicarea și colaborarea între studenți și profesori. Un sistem integrat cu chat și forum-uri de discuții vor permite schimbul rapid de informații, discuții constructive și suport reciproc.

În plus, introducerea notificărilor va veni în ajutorul utilizatorilor să fie mereu la curent. Un planificator integrat pentru organizarea studiului va fi un instrument prin care aceștia își pot gestiona și structura timpul într-un mod mai eficient, și astfel productivitatea va crește.

Referințe bibliografice

- [1] L.R.Pelima, Y.Sukmana, and Y.Rosmansyah. Predicting university student graduation using academic performance and machine learning: a systematic literature review. *IEEE Access*, 2024.
- [2] A.S.Hashim, W.A.Awadh, and A.K.Hamoud. Students performance prediction model based on supervised machine learning algorithms. *IOP conference series: materials science and engineering*, 2020.
- [3] Eibe Frank, Mark A Hall, and Ian H Witten. The weka workbench. online appendix for” data mining: Practical machine learning tools and techniques”. 2016.
- [4] B.H.Kim, E.Vizitei, and V.Ganapathi. Gritnet: Student performance prediction with deep learning. *arXiv preprint arXiv:1804.07405*, 2018.
- [5] H.Zeng and J.Luo. Effectiveness of synchronous and asynchronous online learning: a meta-analysis. *Interactive Learning Environments*, 2024.
- [6] E.Vermeulen and M.LL.Volman. Promoting student engagement in online education: Online learning experiences of dutch university students. *Technology, Knowledge and Learning*, 2024.
- [7] S.Amin et al. An adaptable and personalized framework for top-n course recommendations in online learning. *Scientific Reports*, 2024.
- [8] Bruce Johnson. *Visual Studio Code: End-to-End Editing and Debugging Tools for Web Developers*. John Wiley & Sons, 2019.
- [9] Gabriel Baptista and Francesco Abbruzzese. *Software Architecture with C# 12 and .NET 8: Build enterprise applications using microservices, DevOps, EF Core, and design patterns for Azure*. Packt Publishing Ltd, 2024.
- [10] Anders Hejlsberg, Scott Wiltamuth, and Peter Golde. *The C# programming language*. Adobe Press, 2006.
- [11] Chuck Musciano and Bill Kennedy. *HTML & xhtml: The definitive guide*. ” O’Reilly Media, Inc.”, 2002.

- [12] Hakon Wium Lie and Bert Bos. *Cascading style sheets: Designing for the web*. Addison-Wesley Professional, 2005.
- [13] David Flanagan. *JavaScript: the definitive guide*. ” O’Reilly Media, Inc.”, 2011.
- [14] KR Srinath. Python—the fastest growing programming language. *International Research Journal of Engineering and Technology*, 4(12):354–357, 2017.
- [15] Miguel Grinberg. *Flask web development*. ” O’Reilly Media, Inc.”, 2018.
- [16] John Deacon. Model-view-controller (mvc) architecture. *Online*[[Citado em: 10 de março de 2006.] <http://www.jdl.co.uk/briefings/MVC.pdf>, 28:61, 2009.
- [17] Andrew Lock. *ASP.NET core in Action*. Simon and Schuster, 2023.
- [18] Jon P Smith. *Entity Framework core in action*. Simon and Schuster, 2021.
- [19] Jake Spurlock. *Bootstrap: responsive web development*. ” O’Reilly Media, Inc.”, 2013.
- [20] Ross Mistry and Stacia Misner. *Introducing Microsoft SQL Server 2014*. Microsoft Press, 2014.
- [21] Alan Beaulieu. *Learning SQL: master SQL fundamentals*. O’Reilly Media, 2009.
- [22] Niels Provos and David Mazieres. A future-adaptable password scheme. In *USENIX annual technical conference, FREENIX track*, volume 1999, pages 81–91, 1999.
- [23] Elena Pelican. *Tehnici de optimizare*. Universitatea ”Ovidius” din Constanța, 2025. Note de curs.
- [24] Xia Ning, Christian Desrosiers, and George Karypis. A comprehensive survey of neighborhood-based recommendation methods. *Recommender systems handbook*, pages 37–76, 2015.
- [25] A. Moubayed, M. Injadat, A. B. Nassif, H. Lutfiyya, and A. Shami. E-learning: Challenges and research opportunities using machine learning data analytics. *IEEE Access*, 6:39117–39138, 2018.
- [26] A. Moubayed, M. Injadat, A. Shami, and H. Lutfiyya. Student engagement level in an e-learning environment: Clustering using k-means. *American Journal of Distance Education*, 34(2):137–156, 2020.
- [27] Leo Breiman. Random forests. *Machine learning*, 45:5–32, 2001.
- [28] Elena Pelican. *Algoritmi de calcul științific*. Universitatea ”Ovidius” din Constanța, 2024. Note de curs.
- [29] Douglas C Montgomery, Elizabeth A Peck, and G Geoffrey Vining. *Introduction to linear regression analysis*. John Wiley & Sons, 2021.
- [30] David W Hosmer Jr, Stanley Lemeshow, and Rodney X Sturdivant. *Applied logistic regression*. John Wiley & Sons, 2013.

- [31] Lawrence Chung, Brian Nixon, Eric Yu, and John Mylopoulos. Non-functional requirements. *Software Engineering*, 78, 2000.
- [32] Crenguța Puchianu. *Ingineria Sistemelor Software*. Universitatea "Ovidius" din Constanța, 2024. Note de curs.
- [33] Edgar F Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- [34] Cristina Șerban. *Baze de date*. Universitatea "Ovidius" din Constanța, 2024. Note de curs.
- [35] Peter Pin-Shan Chen. The entity-relationship model—toward a unified view of data. *ACM transactions on database systems (TODS)*, 1(1):9–36, 1976.